



Distributed System Illustrated

A Visual Guide for Engineers

Lichuang

2025

The Charm of Distributed Systems

This is a story about "how to build reliable systems out of unreliable components."

When you open your phone on a rainy night to order food delivery, the system behind it is undergoing complex collaboration: your request passes through a load balancer and gets routed to an instance of the order service; the order data is persisted to the master node of the database and synchronized to two replica nodes across datacenters in milliseconds; at the same time, the inventory service in another cluster is deducting stock, and the payment system is calling a third-party gateway. If any step fails, a series of compensation logics will automatically trigger. The whole process spans dozens of servers, unreliable network links, and hardware that might strike at any time—but in most cases, your food still arrives on time.

This is the daily routine of distributed systems. It looks like magic, but behind it lies decades of the most exquisite engineering practices in computer science.

The Root of Complexity: Confronting Physical Constraints

Transitioning from single-machine to distributed, the most fundamental paradigm shift is: we must learn to coexist with **Partial Failure**.

In a single-machine system, the machine is either running normally or completely dead. But in a distributed system, the system is always in a Schrödinger-like "half-dead" state: a small fraction of nodes crash, or the network partitions, yet the system as a whole keeps running. More fatally, **in an asynchronous network, you can never distinguish whether a node has completely crashed or is merely responding slowly.**

This seemingly simple physical constraint breeds almost all the complexities of distributed systems:

- **Network unreliability:** Messages can be lost, delayed, reordered, or even duplicated. You might think a sent request has succeeded, while in reality, it might be stuck queuing in a router's buffer.
- **Node failures:** Hardware aging, memory errors, OS kernel crashes... any node can unexpectedly go down at any time. What's more troublesome is that it might rejoin the cluster with an inconsistent state after experiencing "amnesia."
- **Absence of a global clock:** The physical clocks of each machine drift at different rates. You cannot simply rely on "the larger timestamp holds the latest state" to resolve conflicts, because in a distributed world, time across different machines is fundamentally incomparable globally.
- **Concurrency is everywhere:** Hundreds of thousands of clients read and write data simultaneously, multiple nodes run for leader concurrently, and Network Partitions cause different parts of the same cluster to act independently. Correctly reasoning about and handling these concurrent behaviors is the most demanding test of fundamental skills in system design.

In the single-machine world, we have an absolutely reliable "source of truth"—the state in memory and disk is deterministic. But in distributed systems, **there is no single source of truth**; each node can only see a local view of the world, like blind men feeling an elephant. How to make these nodes, which only possess local information, piece together a globally consistent state is the core challenge of distributed systems.

The Beauty of Engineering Under Constraints

If distributed systems were merely "difficult," it wouldn't attract so many engineers and researchers to dedicate themselves to it. Its true charm lies in this: facing these seemingly unsolvable physical constraints, humanity has designed extremely elegant solutions.

The **Quorum** mechanism is a great example. Its mathematical foundation is very simple—"any two majority sets must intersect"—but starting from this humble property, we built replication systems that tolerate minority node failures and designed consensus algorithms that guarantee log consistency. Transforming chaotic uncertainty into seamless determinism is itself a process full of intellectual joy.

The **evolution of consensus algorithms** is equally astonishing. Leslie Lamport's **Paxos**

started from a seemingly impossible goal, and through two rounds of simple quorum read-write interactions, provided a rigorous mathematical solution for "how to reach consensus over an unreliable network." The subsequent **Raft** algorithm demystified esoteric theory into an engineering blueprint, allowing strong consistency systems to truly enter the production environments of thousands of households.

The **CAP theorem** demonstrates the beauty of trade-offs in distributed systems, where "you can't have your cake and eat it too." It does not dogmatically prescribe "what can be done," but clearly delineates "what is absolutely impossible." It is exactly these Impossibility Results that establish the boundaries of system design, letting us know where we must compromise and where we can boldly innovate.

This process of finding optimal solutions under severe constraints is the essence of engineering design.

Why Do We Still Need to Learn Distributed Systems in the AI Era?

You might ask: we are now in the age of artificial intelligence, and Large Language Models (LLMs) are reshaping the boundaries of software engineering. Why should we still painstakingly study these foundational distributed theories?

The answer is very simple: modern AI technology is fundamentally built upon the largest-scale distributed systems in human history.

When we talk about hundred-billion-parameter large models, we are no longer talking about algorithms on a single machine, but massive computing clusters composed of tens of thousands of GPUs. In this cluster, the core phantom of distributed systems—Partial Failure—has not disappeared; instead, it is infinitely magnified.

Imagine this: during a model training session spanning months and utilizing tens of thousands of graphics cards, if a certain server's network card experiences microsecond-level jitter, or a specific GPU suddenly suffers a memory fault, how should the entire training task proceed? Without efficient distributed fault-tolerance mechanisms, and without exquisite distributed Snapshot and Checkpoint technologies, training models with hundreds of billions of parameters would be utterly impossible.

Taking a step further, every piece of infrastructure in the AI era has the DNA of distributed systems flowing in its blood:

- Compute scheduling: How do thousands of compute nodes collaborate? Whether it is

data parallelism, tensor parallelism, or pipeline parallelism, they are essentially solving Partition and communication problems in distributed systems.

- **Massive data:** The Vector Databases and distributed object storage that support the AI explosion still rely on Raft or Paxos at the bottom level to guarantee the strong consistency of metadata, and rely on multi-replica Replication to ensure high data availability.
- **Global inference:** When hundreds of millions of users simultaneously initiate conversations with large models, how do we provide low-latency responses globally through distributed routing, load balancing, and distributed sharing of KV Cache? This is exactly the classic battlefield of distributed concurrency and state management.

If neural networks and deep learning algorithms are the "brains" of AI, then distributed systems are the "bones and nerves" supporting this brain's operation. No matter how the upper-layer applications of AI change, as long as it still needs to run on machines in the physical world, it will never escape the cold physical constraints of network latency, node crashes, and physical clock drift.

Therefore, in this era swept by the AI wave, mastering distributed systems is not outdated; on the contrary, it has become the scarcest and most hardcore capability. It allows you not only to remain at the user level of "calling AI APIs," but to possess the confidence of becoming an "AI infrastructure architect," capable of taming those massive computing behemoths that truly drive the future intelligent world.

How This Book is Organized

This book features "illustrations" as its core characteristic, striving to help readers truly understand the underlying principles of distributed systems through intuitive diagrams and logical deduction, rather than rote memorizing dry algorithms. The book's arrangement is not a simple listing of chapters but follows a bottom-up construction logic of "from physical constraints to business abstraction":

- **Chapter 1 Overview of Distributed Systems:** Starting from the definition of distributed systems, we compare them with single-machine systems dimension by dimension, revealing their core challenges: unreliable networks, broken clocks, partial failures, and the resulting data consistency issues. The chapter concludes by discussing the mental shift required from single-machine to distributed thinking—from two-state logic to three-state, from absolute time to relative order, and from perfectionism to the art of trade-offs.

- **Chapter 2 Distributed System Models:** Starting from two classic thought experiments—the Two Generals’ Problem and the Byzantine Generals’ Problem—we establish the theoretical foundation for distributed system design. Subsequently, we define the three core models: communication models (fair-loss links, reliable links, authenticated links), failure models (the hierarchy from crash-stop to Byzantine), and timing models (synchronous, asynchronous, partially synchronous), providing a precise environmental assumption language for the algorithm design in subsequent chapters.
- **Chapter 3 Time and Order in Distributed Systems:** Time is the most fundamental puzzle in distributed systems. This chapter starts with a seemingly simple question—“how to determine the sequence of events in a distributed system”—and gradually reveals why physical clocks are inadequate for this task. We start from the physical principles of quartz and atomic clocks, go through the engineering disasters brought by UTC leap seconds and the flawed symmetry assumptions of the NTP protocol, to deduce the ineliminability of clock skew and drift. Ultimately, we arrive at a surprising conclusion: in distributed systems, “causal order” is more fundamental than “physical time.” Revolving around this insight, the chapter introduces Lamport’s Happens-Before relationship to precisely define causality between events, uses the mathematical language of partial order and total order to explain why certain events are inherently unorderable, and finally derives two logical clocks—Lamport clocks and Vector clocks—as tools to capture causality without relying on a global clock. This chapter also discusses the relationships among state, events, and snapshots, laying the theoretical groundwork for state machine replication and consensus algorithms in subsequent chapters.
- **Chapter ?? Replication:** Replication is the basic means to solve single-point vulnerability. This chapter introduces leader-follower replication (synchronous, asynchronous, semi-synchronous) and quorum read-write mechanisms, systematically outlines the complete consistency model spectrum from linearizability to eventual consistency, and deeply analyzes the CAP theorem and its improved form PACELC, elucidating the trade-off between consistency and availability.
- **Chapter ?? Distributed Consensus Algorithms:** Starting from the FLP impossibility theorem, we progressively deduce the core ideas of the Paxos algorithm evolving from quorum read-writes, and then detail the complete design of the Raft algorithm—leader election, log replication, safety guarantees, membership changes, liveness optimization, and log compaction. This is the capstone chapter of the entire book.
- **Chapter ?? Partitioning:** If replication is for fault tolerance, partitioning is for scalability.

This chapter explains data sharding techniques, including engineering practices like consistent hashing principles and virtual node optimization, solving the horizontal scaling problem when single-machine capacity is insufficient to hold all data.

- **Chapter ?? Transactions:** After mastering the underlying replication and partitioning, we will return to the application perspective. Starting from single-machine ACID properties, this chapter analyzes Undo/Redo log mechanisms and concurrency control (2PL, OCC, MVCC, Write Skew), extending to distributed transaction solutions like 2PC/3PC, Google Spanner's TrueTime innovation, and flexible transaction schemes like TCC and SAGA.

It is recommended to read in order for the first time, as advanced architectures are always built upon solid cornerstones. Later chapters will frequently reference earlier concepts.

Who This Book is For

This book is suitable for the following readers:

- Backend engineers and architects with some programming experience, eager to break through their technical ceilings and dive deep into the distributed foundations.
- College students studying distributed systems courses, hoping to find a practical reference book with more engineering intuition than traditional textbooks and more accessibility than academic papers.
- R&D personnel preparing for System Design interviews, who need to build a solid understanding of core algorithms and consistency models.
- Any tech geek curious about "how large-scale internet systems operate."

This book assumes readers have basic computer science literacy (such as data structures, operating system principles, computer networks), but does not require prior knowledge of distributed systems. When encountering necessary mathematical and theoretical concepts, we will explain them thoroughly and accessibly within the context.

Final Words

Distributed systems is a field where theory and engineering are highly intertwined. Algorithms in papers often have elegant proofs, but in real production environments, we must face various realistic failure issues like network jitter, silent disk errors, and even datacenter power outages. There is a massive chasm between deriving the Paxos algorithm on a whiteboard and actually writing industrial-grade code that can withstand the tests of a production environment.

The original intention of writing this book is to build a bridge over this chasm. I try to filter out overly obscure academic expressions, using the language engineers are familiar with and intuitive diagrams to peel back and thoroughly explain those time-tested classic principles layer by layer.

In this field, there is no universally applicable "silver bullet," only trade-offs and choices based on profound understanding. If, while reading the source code of an open-source project, you can suddenly see the light because of a diagram in this book; or when facing complex system architecture designs, you can have a bit more confidence in moving from ideal to implementation, then my goal in writing this book will have been achieved.

The road to exploring distributed systems is full of challenges, but equally full of joy. I hope this little book can become your steadfast and reliable companion on this journey.

Lichuang
2025

Contents

The Charm of Distributed Systems	i
1 Overview of Distributed Systems	1
1.1 What Is a Distributed System	1
1.2 Challenges of Distributed Systems	5
1.2.1 Unreliable Networks	7
1.2.2 Clock and Ordering Problems	8
1.2.3 Partial Failure	10
1.2.4 Data Consistency	11
1.3 Mindset Shift	14
1.4 Chapter Summary	16
2 Models of Distributed Systems	18
2.1 Two Generals' Problem	18
2.2 Byzantine Generals' Problem	25
2.3 System Models	27
2.3.1 Communication Models	28
2.3.2 Failure Models	30
2.3.2.1 Crash Model	31
2.3.2.2 Omission Model	33
2.3.2.3 Byzantine Model	34
2.3.3 Timing Models	36
2.4 Chapter Summary	38
3 Time and Order in Distributed Systems	40
3.1 State, Events, and Snapshots	41
3.2 Physical Clocks	44
3.2.1 Physical Time Sources and Representation	44
3.2.2 Wall Clocks and Monotonic Clocks	47
3.2.3 Physical Time Synchronization	48
3.2.4 Deficiencies of Physical Clocks	51

3.3	Causality and Event Ordering	53
3.4	Total Order and Partial Order	58
3.5	Causality and the Happened-before Relation	62
3.6	Logical Clocks	64
3.6.1	Lamport Clocks	65
3.6.2	Vector Clocks	69
3.7	Global Snapshots in Distributed Systems	73
3.8	Chapter Summary	81
	Bibliography	83

Overview of Distributed Systems

In the evolution of modern software engineering, the transition from single-node applications to distributed architectures represents a critical watershed. This leap is not merely a matter of hardware stacking or code migration, but a profound transformation involving shifts in mindset, design philosophy, and even a renewed understanding of the laws of physics.

As the opening chapter of this book, it will lead readers out of the comfort zone of single-node systems and confront the real challenges of distributed environments. We will first clarify the core definition of distributed systems and analyze their essential differences and advantages over centralized systems. Next, we will focus on the unavoidable technical problems in this field—from unreliable network communication to fragmented global clocks, and the uncertainty brought by partial failures. Finally, this chapter will elaborate on the mindset shifts architects must undergo during this transition: moving from pursuing absolute certainty to seeking a balanced trade-off between consistency and availability. Understanding these fundamental concepts and paradigms is a prerequisite for mastering the complex algorithms and consistency models in subsequent chapters, and it is also the cornerstone of building highly reliable and scalable systems.

1.1 What Is a Distributed System

Before diving into distributed systems, let us first look at their opposite: the centralized **single-node system**, which is also sometimes referred to as a *monolithic system* because all computation,

storage, and processing tasks are concentrated on a single computer or a central node. The main characteristics of single-node systems are:

- **Single node:** All functions (such as computation, data storage, and user request processing) are completed on one machine or a single server.
- **Shared resources:** Shared memory and local resources are used, communication is fast, and no network message passing is required.
- **Single point of control:** All decisions and data management are controlled by the central node, with no need for inter-node coordination.
- **Single clock:** All running processes on the machine use the same clock time.

The advantages of centralized single-node systems are their simplicity: requests, computation, storage, and processing are all completed on the same node, so issues unique to distributed systems such as network latency and inconsistent system time do not exist. However, the disadvantages are also obvious: the failure of a single node can cause the entire system to become unavailable, lacking fault tolerance; at the same time, performance is limited by the processing capacity of a single node and can only be improved by upgrading hardware (*vertical scaling—scale up*, see Figure 1.1).

Examples of centralized systems include:

- Traditional single-node databases.
- Single-server web applications that handle all requests and data storage.
- Local applications running on a personal computer.

However, many application scenarios and requirements cannot be solved by centralized systems, for example,

- **High performance:** Centralized systems can improve hardware performance through *vertical scaling* (using faster CPUs, larger memory, etc.) to enhance processing capacity. However, vertical scaling can never break through the physical ceiling of a single node. In contrast, distributed systems use (*horizontal scaling—scale out*, see Figure 1.1) by combining multiple machines into a system to provide services externally. When the current system still does not meet requirements, more machines can be added to improve processing capacity. For example, in the 2003 paper [1], Google described their experience building a distributed file system using inexpensive hardware.
- **High scalability:** Like the performance bottleneck, as data volume grows, centralized systems also encounter bottlenecks in data storage. Distributed systems, by distributing data across multiple machines, can increase nodes to provide greater storage capacity

when the system can no longer handle more data.

- **High availability:** The vast majority of services today require systems to run 24/7. In centralized systems, only one machine node provides service, which easily leads to the *single point of failure* problem: the failure of one machine causes the entire system to become unavailable. In distributed systems, multiple machines form a system, providing more redundancy so that the system can continue to run even when some machines fail.
- **Geographic proximity to users:** In many cases, the geographic location of users must be considered. For example, a social application has users from both Asia and Europe; in such cases, services may need to be established on different continents. These services distributed across different regions need to coordinate: for example, user IDs must be globally unique.
- **Compliance requirements:** With increasingly strict global data privacy regulations, system architectures must support cross-regional distributed deployment to meet compliance requirements for *data residency* and *data sovereignty*. By establishing independent data center nodes in different countries or jurisdictions, localized storage and processing of data is achieved, ensuring that user data is strictly confined within its geographic boundaries.
- **Resource sharing:** The term “resources” here refers not only to hardware resources but also to software resources. For example, when performing large amounts of computation, to speed up the process, the computation needs to be divided into multiple tasks and distributed to multiple machines for simultaneous execution. Through products like online shared documents, multiple users can browse and modify documents simultaneously without downloading files to their hard drives for editing, and no longer need to install processing software on their personal computers.
- **Business attributes:** Finally, beyond the physical limitations of centralized systems that require distributed architectures, certain businesses inherently require distributed characteristics. For example, in an e-commerce service, browsing and purchasing products happen on one platform, but paying the bill must be completed on another system (such as a payment platform or bank). In such businesses, the system must naturally be distributed.

With the horizontal comparison to single-node systems, we can now look at the definition of distributed systems. This book adopts the definition of distributed systems mentioned in [3]:

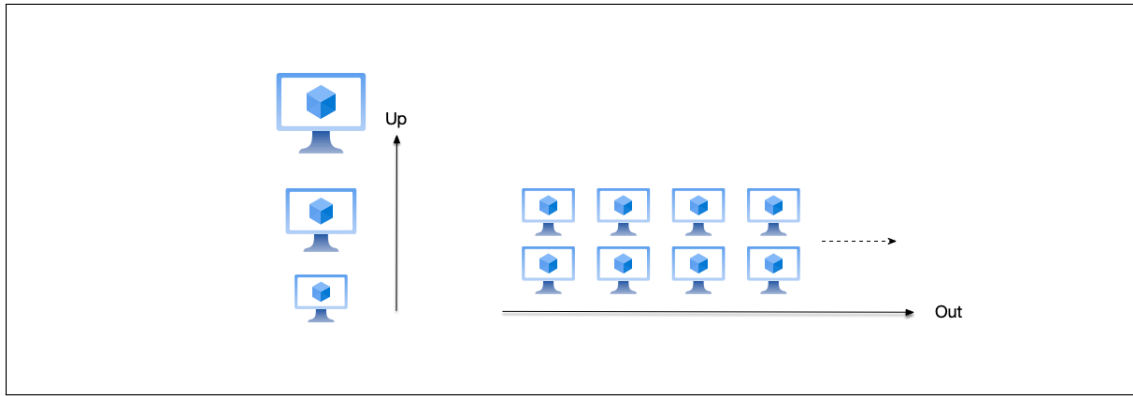


Figure 1.1: Vertical scaling improves the processing capacity of a single-node system by enhancing the performance of a single node (using faster CPUs, larger memory, and larger hard drives), while horizontal scaling improves the processing capacity of the system by adding more machines.

A distributed system is a system whose components are located on different networked computers, which communicate and coordinate their actions by passing messages to one another.

From the above definition, we can see that a **Distributed System** is a system composed of a set of independent computers (nodes) connected through a network, which collaborate to complete common tasks. Each node has its own processor, memory, and storage. Nodes communicate through message passing, with no shared memory.

Each computer that makes up a distributed system is called a *node*. Here, the concept of “computer” is very broad: it could be a personal computer, a server in a data center, a mobile device, or various types of IoT devices connected to the network (such as cars, smart home appliances, etc.). In short, a “node” can be any device capable of network communication.

Distributed systems aim to achieve high availability, scalability, and fault tolerance. Distributed systems typically have the following characteristics:

- **Multi-node collaboration:** A distributed system consists of multiple physically dispersed nodes working together, possibly located in different geographic locations or as independent processes. The address spaces of nodes are independent, so shared-memory multiprocessors are not considered representative of distributed systems.
- **Network communication:** Nodes communicate through a network (such as the Internet or a local area network). Network message latency depends on the specific conditions of the network link. For example, fiber optic cables may be severed, networks may be partitioned, and message latency may suddenly spike—these are all common failure

scenarios in distributed communication.

- No global clock: Nodes lack unified time and need to synchronize through protocols.
- Collaborative work: Multiple nodes that make up a distributed system communicate and coordinate to achieve a common goal. For example, a storage system composed of three nodes jointly provides strongly consistent data services and can continue to work even when one node fails. Another example is dividing a large task into multiple small tasks, executing them jointly by multiple nodes, and then aggregating the results to obtain the final outcome.

Table 1.1 and Figure 1.2 list the key feature comparisons between distributed systems and centralized systems.

Feature	Distributed System	Centralized System
Architecture	Multi-node, connected through network	Single node
Communication	Message passing through network, latency unbounded	Local call, low latency
Scalability	Horizontal scaling	Vertical scaling
Fault tolerance	High fault tolerance through multi-node redundancy	Single point of failure affects availability
Consistency	Need to define the allowed consistency model	Naturally consistent, no extra coordination needed
Complexity	Complex design and maintenance	Simple design and maintenance

Table 1.1: Comparison between centralized systems and distributed systems

However, although distributed systems have many advantages over centralized systems, implementing a reliable distributed system is not easy. Let us proceed to look at the challenges of implementing distributed systems.

1.2 Challenges of Distributed Systems

When developers first turn to distributed system development, they bring with them much of the inherent experience from single-node system development. L. Peter Deutsch and others, during their time at Sun Microsystems, summarized the fallacies of distributed computing from

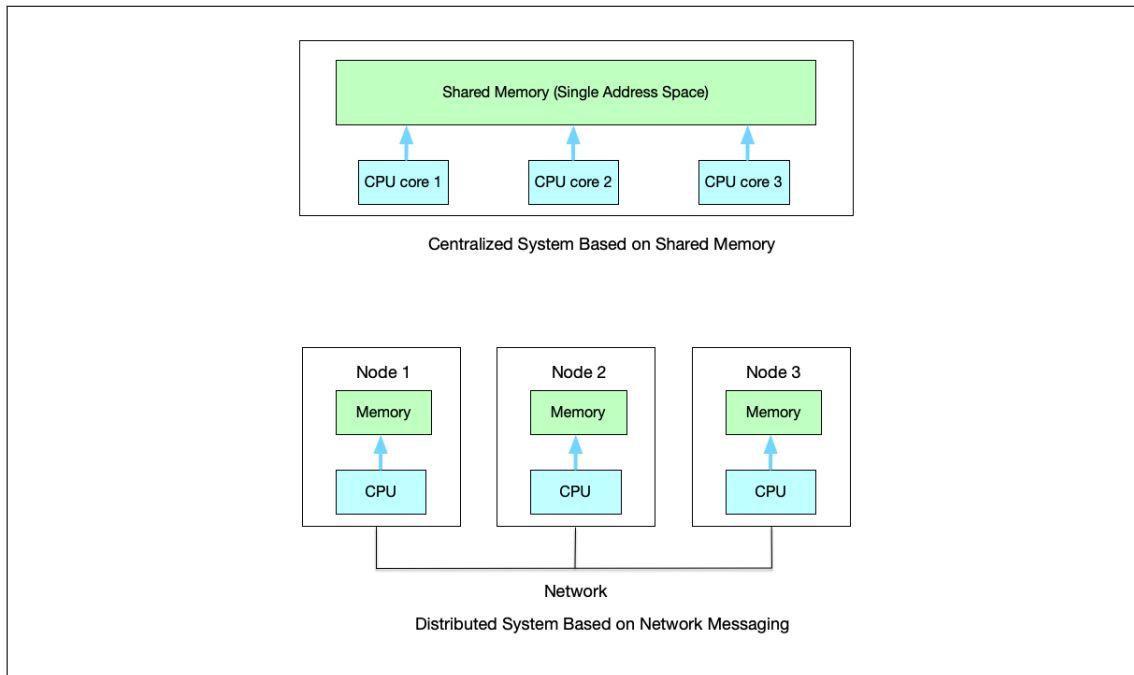


Figure 1.2: Architecture comparison between distributed and centralized systems

their work¹:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology doesn't change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

We will discuss some of these issues in detail below.

¹https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing

1.2.1 Unreliable Networks

In single-node systems, messages are all local calls, with low latency and no possibility of non-delivery. In single-node programs, communication between two modules (such as function A calling function B) is completed through the memory bus and CPU instructions, which guarantees the following characteristics for communication under single-node systems:

- **Reliability:** Barring hardware failures, anomalies like data loss rarely occur.
- **Synchronicity:** This communication is usually synchronous.
- **Bandwidth and latency:** Bandwidth is extremely wide, latency is extremely low (nanosecond level), and very stable.

In distributed systems, however, nodes communicate through the network, and network communication often has unbounded latency: that is, it cannot guarantee any bound on data latency, and in some cases, messages may not arrive at all. For example, message delivery in the network may exhibit the following phenomena:

- **Message loss:** Requests may be dropped during router queuing, or because fiber optic cables are accidentally severed (case (a) in Figure 1.3).
- **Different messages arrive in a different order than when sent, or different nodes receive messages in different orders.** When the same two nodes communicate, it is possible that message 1 was sent first and message 2 was sent later, but because they took different routing paths, message 2 may arrive before message 1.
- **Due to network partitions and other factors, messages cannot be received by the peer, even though the receiving node is still in a working state.**
- **Due to message sending failures, messages may be retransmitted, causing the receiving node to receive multiple identical messages.**
- **In a single node, the time to read memory has an upper bound. But in a network, a data packet may be delayed by 1 ms or 1 minute, and it is impossible to distinguish whether the receiving party has crashed (case (b) in Figure 1.3), is processing slowly, or there is a network problem.**
- **The message is successfully processed by the receiver and acknowledged, but the acknowledgment message is lost on the way back (case (c) in Figure 1.3).**
- ...

As can be seen, when the above failures occur, they manifest as message delays, out-of-order delivery, and loss, but it is impossible to distinguish whether it is a “node failure (such as a crash)” or a “network failure.”

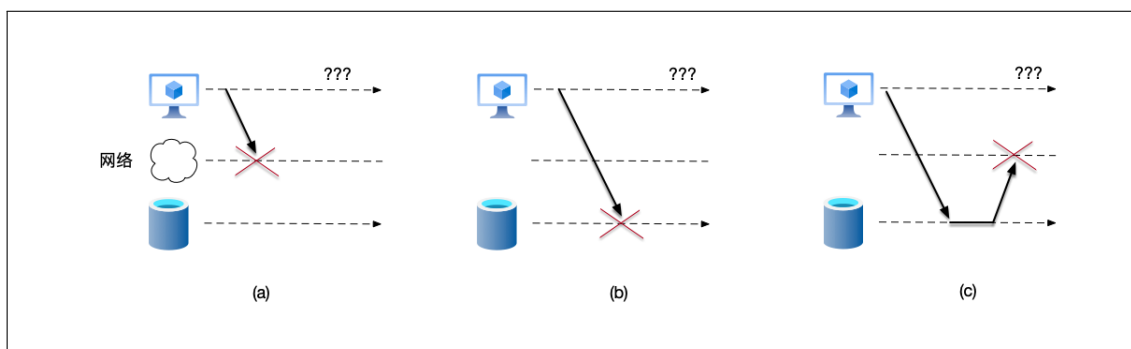


Figure 1.3: In distributed systems, various failures may occur during message delivery: (a) message is lost during sending; (b) message encounters an error during processing; (c) message is lost during reply. From the receiver's perspective, these failures all appear as not having received a reply, and the specific cause cannot be distinguished.

Therefore, even when using a transport protocol like TCP that encapsulates error retry and congestion control logic, developers cannot assume the network is reliable. TCP indeed performs retransmission and sequencing at the transport layer, providing the application layer with an illusion of a “reliable stream.” However:

- TCP cannot solve the problem of connection disconnection: if the network cable is unplugged, TCP will retry several times and eventually report an error. At this point, the application layer still has to handle the failure.
- TCP masks latency: to ensure ordering, TCP will block subsequent data packets, causing the application layer to experience severe latency jitter.
- TCP cannot guarantee “delivery” at the business level: a TCP ACK only means the kernel is telling the kernel “I received it,” not that the application has processed the request. For example, it is possible that after the data packet is received by the kernel but before it is processed by the application, the application crashes.

1.2.2 Clock and Ordering Problems

In human intuition, time is linear and absolute. But in distributed systems, time is fragmented. In single-node systems, there is only one source of time, and the operating system is like an absolute referee holding a stopwatch. for example,

- Process A writes a file at 12:00:01.
- Process B reads the file at 12:00:02.
- Undoubtedly, A happened before B.

Because single-node systems have only one global clock, physical timestamps can be safely used to determine the order of events. As shown on the left side of Figure 1.4, if you see in the logs that the timestamp of event X is smaller than that of event Y, then X definitely happened before Y. In the single-node world, the unique system-wide global clock provides a unique, strict sequence for all events.

However, in distributed systems, due to the following factors, there is no globally unique clock:

- Different nodes have different times. For example, the time on node A may be one second faster than the time on node B, so the order of events cannot be determined by simply comparing the times on different machines.
- Messages are transmitted between nodes through network communication, and network latency fluctuates, making it impossible to predict how long message delivery actually takes.

The above facts make it difficult in distributed systems to determine the order of events involving multiple machines based on the physical time of nodes. As shown on the right side of Figure 1.4:

- Node A, with a faster clock, sends a message at time 10:01:54.765.
- Node B, with a slower clock, receives this message, and its local time is now 10:01:54.432.
- From the logs, B received the message before A sent it, which violates the laws of physics. Node B seems to have received a message from the future.

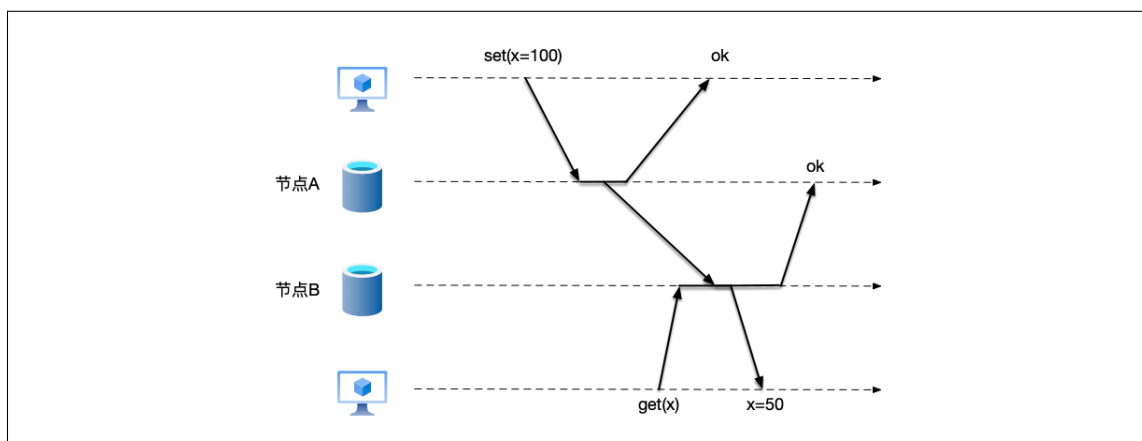


Figure 1.4: single-node systems have a unified global clock, so events on a single-node system have a unified order. In distributed systems, different nodes maintain their own time. If local physical time is used to judge the order of events, it may result in receiving a message from the “future.”

Therefore, in distributed systems, if physical timestamps are relied upon to resolve conflicts (such as “whoever writes last wins”), this clock error can lead to catastrophic data overwrites. We will explore the issues of time and ordering in distributed systems in depth in Chapter 3.

1.2.3 Partial Failure

In single-node systems, system behavior is *deterministic*, existing in only two possible states: either working normally or crashed. This “*all-or-nothing*” binary state is relatively easy to handle:

- If the code logic is wrong, the program will throw an exception or report an error.
- If the hardware fails (such as a power outage), the entire operating system will stop, and the program will terminate abruptly.

But in distributed systems, where multiple nodes coordinate work through network communication, they face a completely new failure mode: *Partial Failure*. Partial failure refers to a situation where some nodes or parts of the network in the system have failed, while other parts continue to run normally. “Partial failure” is far more difficult to handle than “complete crash” because it introduces *uncertainty* (*nondeterminism*).

Imagine that service A sends a request to service B, wanting to deduct 100 dollars from the balance. In a single-node function call, this would quickly return success or failure. But in a distributed network, service A waits for 5 seconds and receives nothing. At this point, the following possibilities exist, and service A cannot distinguish between them:

- Request lost: The request never reached service B due to network failures.
- Node crash: The request reached service B, which was halfway through processing, or had just deducted the money but had not yet had time to reply, when B’s power cord was unplugged.
- Response lost: Service B successfully processed the request, the deduction was successful, and it sent back “OK,” but the “OK” packet was lost on the way back.
- Slow processing: Service B is alive and well, but is responding slowly due to a garbage collection pause or database lock contention was too fierce, and it has not yet had time to reply.

This state of “not knowing what happened, so not knowing what to do” is the greatest challenge brought by partial failure. We will explain in detail the matters related to detecting node failures in Section ??.

Specific problems brought by partial failure include:

- “Sub-healthy” processes: A failing node may enter a “zombie” or degraded state. It occa-

sionally responds and occasionally times out, causing the caller to not dare to disconnect easily and to keep retrying.

- **Data inconsistency:** If the request is to “deduct money from a bank account” and the response is lost, the sender, assuming failure, initiates a retry—even though the receiver has already successfully deducted the money. Without special handling, this may lead to duplicate deductions.
- **Resource exhaustion (cascading failure):** This is the most fatal. Suppose service A depends on service B. Service B experiences partial failure (responses become extremely slow). Service A’s threads will block waiting for B’s response. As requests continue to pour in, all of service A’s threads will be suspended waiting, causing service A’s memory and CPU to be exhausted, and eventually service A also fails. This is like a car getting a flat tire and slowing down (partial failure) on a highway, causing the entire highway behind it to be jammed (cascading failure).
- **Ghost behavior:** Sometimes the system determines that a node is down and offline (because network congestion prevented a heartbeat), and so it starts a standby node. As it turns out, the original node did not crash, but was just slow to respond for some reason. When the network recovers, two nodes operate on the same data simultaneously (split-brain), causing data corruption.

It is precisely for these reasons that Leslie Lamport once wryly described distributed systems as:

A distributed system is one in which the failure of a computer you didn’t even know existed can render your own computer unusable.

Although this is not a formal definition of distributed systems, it captures the challenges faced by distributed systems. In short, when a single-node system fails, it is easy to know that the system has a problem, but in distributed systems, the phenomenon is message timeout, and yet it is impossible to know exactly what happened.

1.2.4 Data Consistency

If “partial failure” represents the inherent physical flaw of distributed systems, then the “data consistency” challenge constitutes its core logical contradiction. To thoroughly understand the complexity of distributed architectures, it is necessary to first review the idealized environment provided by the single-node model. In that world, strong consistency is an innate fundamental

guarantee:

- Shared memory model: In multi-threaded programs, as long as a lock is added or atomic variables are used, when thread A modifies variable x to 100, thread B reading x afterward will definitely read 100.
- Single database: In a MySQL instance running on a single node, once a transaction is committed, the data is persisted. No matter who queries afterward, they will see the latest data.

In the single-node world, there is only one truth, and the propagation of truth is instantaneous. There is an invisible “global observer” (the operating system or database kernel) that guarantees all observers always see the same state.

But in distributed systems, when multiple machines communicate through network messages, the laws of physics (the limit of the speed of light) shatter this beautiful illusion. To prevent single points of failure, we usually replicate data to multiple machines (such as in a master-slave architecture). Once replicas are introduced, the single truth ceases to exist; instead, multiple divergent states may emerge. As shown in Figure 1.5, imagine the following scenario:

1. Node A receives a user’s write request: “Change the balance to 100.”
2. After updating its own data, node A responds to the client that the write was successful.
3. Node A notifies node B through the network: “Change the data to 100.”
4. During the few milliseconds (or even seconds) of delay in this network transmission, another user sends a query request to node B but receives old data.

At this point, node A says the balance is 100, and node B says the balance is 0. Who is right?

- From a temporal perspective, the data on node A is the latest.
- But from node B’s perspective, it has no idea what happened at A.

This is the consistency problem of distributed systems: due to the delay in message communication, there is no other way to ensure that all nodes have the same data view at the same moment.

Fortunately, in distributed systems, consistency is not a black-and-white Boolean value (yes/no), but a *spectrum*. The architect’s job is no longer to find “perfect truth,” but to find the subtle balance point between “how long to make users wait for data accuracy” (latency/strong consistency) and “how much old data to allow users to see for speed” (availability/eventual consistency) based on business scenarios. We will discuss various consistency models in Section ?? and distributed consensus algorithms in Chapter ??.

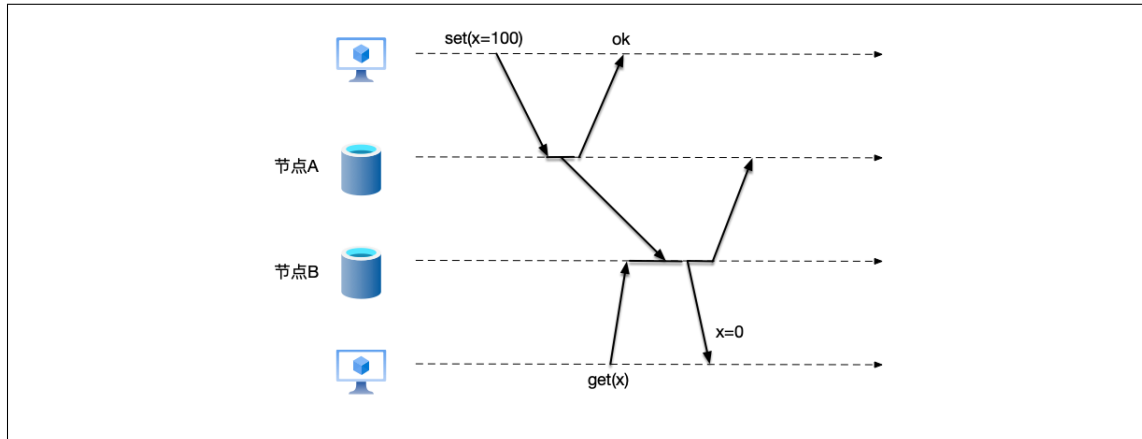


Figure 1.5: In a distributed system, due to message delays, data on two nodes becomes inconsistent, and different clients accessing different nodes get different results.

The above are the challenges faced by distributed systems. The remaining content of this book will revolve around these challenges, explaining in detail how to solve these problems. Table 1.2 provides a simple comparison and summary. We will begin our discussion of this book with these questions.

Dimension	Single-node Features	Distributed Challenges	Key Technologies/Solutions
Time	Physical clock total order	Physical clock unreliable (3.2)	Logical clocks (3.6), TrueTime (??)
Failure	Crash means stop	Partial failure	Replication (Ch. ??), consensus (Ch. ??)
Consistency	Strong consistency (ACID)	Replica delay, data conflicts	Consistency models (Sec. ??), distributed transactions (Ch. ??), consensus (Ch. ??)
Capacity	Single-node hardware limit	Unlimited data volume	Partitioning (Ch. ??)

Table 1.2: Characteristics of centralized systems and challenges of distributed systems

1.3 Mindset Shift

We have already understood the differences between single-node and distributed systems. When engineers transition from single-node system development to distributed system development, the following mindset shifts are required:

From “Binary Logic” to “Ternary Logic”

In single-node systems, a function call has only two possible outcomes: either success (returning a result) or failure (throwing an exception). In single-node systems, computers are reliable; CPU instructions either execute or they do not.

In distributed systems, *remote procedure calls (RPC)* between nodes have three possible outcomes: in addition to success and failure, there is also the possibility of message timeout. When message timeout occurs, there is no way to know whether the other party did not receive the request, is processing slowly, or the reply was lost after processing.

Therefore, distributed engineers must learn to handle such a “Schrödinger state.” In message communication, *idempotency* design must be introduced: because it is not known whether the last attempt succeeded, the system must be designed to allow retries and ensure that the result of multiple retries is the same as a single attempt.

From “Global Consistency” to “Relativity”

In single-node systems, because there is a globally unique clock, event times have a strict order. For example, if logs show that event A occurred at 10:00:01 and event B occurred at 10:00:02, then event A definitely occurred before event B.

But in distributed systems, because there is no globally unique clock, the reliance on “absolute time” must be abandoned, and instead the “order of events” must be focused on. In addition, engineers need to accept eventual consistency in certain scenarios: at this moment, the balance is 100 dollars from service A’s perspective, while it is 90 dollars from service B’s. This is “normal” in distributed systems, as long as we reach agreement after a while.

From “Perfectionism” to “The Art of Trade-offs”

In single-node systems, ACID (Atomicity, Consistency, Isolation, Durability) is regarded as the golden standard taken for granted. The database promises: as long as a transaction is committed, the data will never be lost, and everyone will see the latest state. Developers are accustomed to this “perfect” guarantee.

However, in the distributed world, the CAP theorem (Consistency, Availability, Partition

Tolerance) shatters this illusion. Beginners often mistakenly think that CAP means you can arbitrarily pick two out of C, A, and P, but in distributed reality, partition tolerance is not an option but an objective fact that must be accepted. Network partitions are inevitable: fiber optic cables will be severed, routers will be congested, and even data centers may lose power. Once a network partition occurs (nodes cannot communicate with each other), the system must face a cruel choice:

- If you want 100% availability (ability to read and write at any time), you must sacrifice strong consistency (possibly reading stale data).
- If you want strong consistency (everyone must see the same thing), you must sacrifice availability during a network partition (system reports an error or blocks).

This requires engineers to no longer seek a “perfect solution,” but to find “the most suitable solution for the business.” In distributed architecture, there is no best design, only the most reasonable trade-off under specific constraints.



We will discuss ACID in depth in Section ?? and the CAP theorem in Section ??.

From “Defending Against Failures” to “Embracing Failures”

In single-node systems, failures are exceptional cases, and engineers strive to avoid system crashes.

But in distributed systems, failures are the norm:

- When a system has 1,000 machines, hard drives may fail every day, and network jitters may occur at every moment.
- Special attention must be paid to partial failure: the entire system has not crashed, but because some non-critical service is responding slowly, the entire thread pool is dragged down, causing the main site to go down.

This requires engineers to design for failure—assuming dependencies will inevitably crash, delay, or deliver out-of-order messages. By adopting patterns like circuit breakers, graceful degradation, and bulkheads, systems can isolate the failing components rather than cascading into total failure.

1.4 Chapter Summary

This opening chapter officially unlocks the door to distributed systems. Starting from the most basic definitions, and through comparison with traditional centralized single-node systems, we have established a fundamental understanding of distributed systems and analyzed the complex challenges and paradigm shifts behind them.

1. Architectural evolution: from single-node to distributed. Single-node systems are based on shared memory and a single clock, with advantages such as simple models and strong consistency, but face single points of failure and the physical bottleneck of vertical scaling. Distributed systems are composed of a set of independent computers (nodes) interconnected through a network, which communicate and coordinate through message passing. To break through the limitations of single nodes, we have turned to distributed architectures. Through horizontal scaling, using the collaboration of multiple machines, we have achieved high performance, high scalability, and high availability, and met the needs for data compliance and business isolation.
2. Core challenges: beautiful illusions are shattered. Distributed systems are powerful, but they are not a free lunch. We have shattered common myths such as “the network is reliable” and “latency is zero” (fallacies of distributed computing) and confronted the following major core challenges:
 - Unreliable networks: Unlike the reliable memory bus of a single node, network communication has problems such as packet loss, out-of-order delivery, and unbounded latency. Even the TCP protocol cannot solve business-level “delivery” confirmation or logical handling after connection disconnection.
 - Clocks and ordering: Distributed systems lack a globally unique physical clock, making it impossible to rely solely on physical timestamps to determine the order of distributed events; new logical clock mechanisms must be found.
 - Partial failure: This is the most tricky problem in distributed systems. Unlike the definite state of “all or nothing” in single-node systems, distributed systems have a “zombie” state—it is impossible to determine whether the other party is dead or alive, and impossible to determine whether a request was executed successfully.
 - Data consistency: Due to the existence of data replicas and the physical limit of the speed of light, it is extremely difficult to ensure that all nodes have the same data

view at the same moment. We need to find a balance between “strong consistency (waiting for synchronization)” and “availability (fast response).”

3. Mindset shift: the engineer’s self-evolution. Transitioning from developing single-node applications to distributed systems is not only a technical upgrade but also a reconstruction of mindset:

- From binary to ternary: Abandon the “success/failure” binary and learn to handle the “success/failure/timeout (unknown)” ternary logic, and introduce idempotency design.
- From absolute to relative: Abandon reliance on global absolute time and instead focus on the causal order of events and logical clocks.
- From perfection to trade-offs: Accept the constraints of the CAP theorem, no longer seek perfect strong consistency, but make trade-offs between consistency and availability based on business scenarios.
- From defense to embracing failures: Admit that failures are the norm, and design fault-tolerant systems through techniques such as circuit breakers and degradation, rather than praying that failures do not occur.

Through the study of this chapter, readers have gained a preliminary understanding of distributed systems and know the challenges and difficulties of distributed systems. In the following chapters, we will provide specific engineering solutions and theoretical foundations for the issues raised in this chapter—replication, partitioning, logical clocks, consensus algorithms, consistency models—one by one.

Models of Distributed Systems

In Chapter 1, we compared single-node systems with distributed systems. A single-node system communicates through shared memory, has a globally unique clock, and exhibits deterministic behavior when errors occur, making programming on such systems relatively straightforward. In contrast, a distributed system consists of multiple nodes that communicate via messages, which makes distributed systems significantly more complex. Therefore, before designing a distributed system, we need a theoretical framework to describe how the system operates. Designers must clarify the assumptions and conditions of the runtime environment at design time, as different conditions entail varying implementation challenges. *Models of distributed systems* are the theoretical frameworks used to describe and analyze the behavior, properties, and design of distributed systems. They provide an abstraction for studying communication, computation, failures, and synchronization in distributed systems, helping designers and researchers understand system behavior and solve practical problems in complex environments.

In this chapter, we start our discussion of distributed-system models with two famous thought experiments: the Two Generals' Problem and the Byzantine Generals' Problem.

2.1 Two Generals' Problem

*The Two Generals' Problem*¹ is a thought experiment in the field of distributed systems. It aims to illustrate the pitfalls and challenges of coordinating operations among multiple nodes in a

¹https://en.wikipedia.org/wiki/Two_Generals%27_Problem

distributed system through an *unreliable connection*.

The experiment assumes two armies, each led by a general. The two armies are preparing to attack a city. The city's defenses are strong, and both armies must act together to win; either army acting alone will fail. The two generals need to coordinate a common time to attack. The two armies are stationed on opposite sides of the city, separated by a valley. The only way for them to communicate is to send messengers across the valley. However, the valley is controlled by the city's defenders, so messages passing through it may be captured.

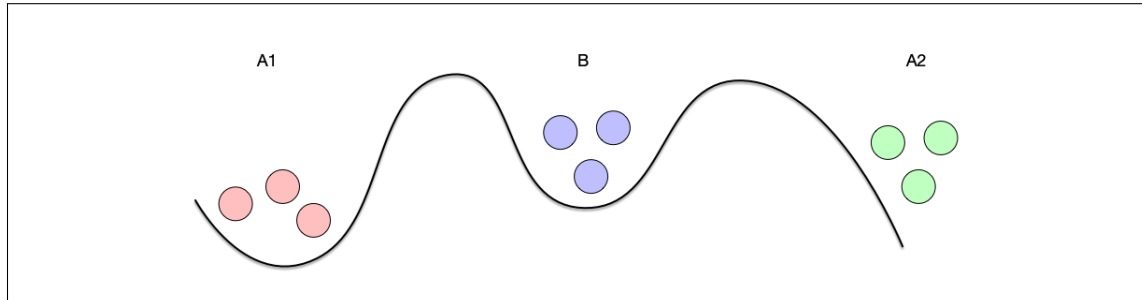


Figure 2.1: Armies A1 and A2 cannot see each other directly; they can only communicate through messengers, but their messengers may be captured by Army B.

Although the two generals have already agreed to attack the city, they have not yet agreed on the time of the attack. Both generals must order their armies to attack simultaneously in order to win. Therefore, they must communicate and coordinate a time to attack. If one general believes a time has been coordinated while the other is unaware, and only one army attacks, the result will be a disastrous failure, as shown in Table 2.1:

Army A1	Army A2	Result
Do not attack	Do not attack	Nothing happens
Attack	Do not attack	Army A1 fails
Do not attack	Attack	Army A2 fails
Attack	Attack	City captured

Table 2.1: Possible outcomes of launching an attack

In this experiment, the “generals” are the nodes in a distributed system, and the messengers that communicate between them are the connections between nodes. We can see that these connections are unreliable: messages may be lost. Let us examine whether the two generals can reach consensus on an attack time when communicating over such an unreliable connection.

Suppose General A1 sends a message to General A2 saying, “Attack at 09:00 on August 4.” Because the messenger may be captured, General A1 is uncertain whether the message will reach General A2. This uncertainty makes A1 hesitate, because if his army attacks alone, it will face a disastrous defeat.

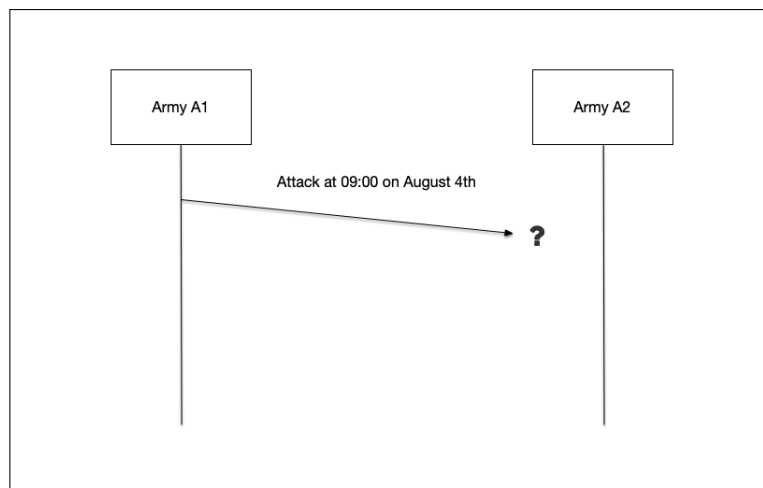


Figure 2.2: Army A1 is unsure whether the message coordinating the attack time will reach Army A2.

To eliminate this uncertainty, A1 and A2 agree that after receiving a message, the receiver will send an acknowledgment so that the sender can confirm the message was successfully received. However, the acknowledgment may also be captured, so it is also uncertain whether it will reach the receiver. This uncertainty makes A2 hesitate, because if his army attacks alone, it will face a disastrous defeat.

At this point the problem reaches a dead end: no matter how many acknowledgment messages are sent, neither general can have sufficient reason to believe that their messenger has not been captured by the enemy.

Based on the scenarios above, the Two Generals' Problem highlights the following difficulties:

- **Uncertainty of message delivery:** General A1 sends a message to General A2 telling him the attack time, but cannot confirm whether A2 received it. Even if A2 receives it and replies with an acknowledgment, A1 cannot confirm whether the reply was delivered. This acknowledgment process may loop infinitely, because every acknowledgment requires a further acknowledgment.
- **Infinite-acknowledgment problem:** To ensure both sides agree on the attack time, A1 needs an acknowledgment from A2, A2 needs an acknowledgment from A1 that A2's

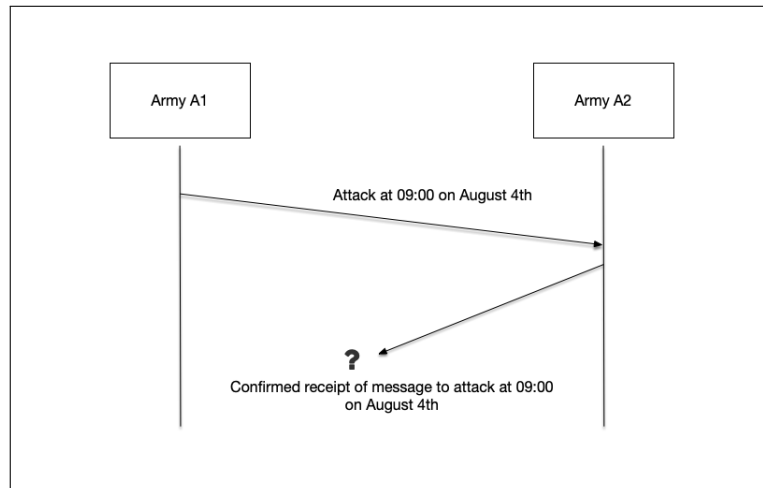


Figure 2.3: Army A2 is unsure whether the acknowledgment of the coordinated attack time will reach Army A1.

acknowledgment was received, and so on. Because communication is unreliable, each acknowledgment may be lost, making it impossible to reach definite consensus.

The Two Generals' Problem was first proposed by E. A. Akkoyunlu, K. Ekanadham, and R. V. Huber in a 1975 paper[5], which described communication between two gangs and gave a proof that this class of problems is unsolvable. In 1978, Jim Gray named it the “*Two Generals Paradox*” in his book[6].

The Two Generals' Problem shows that in an *unreliable communication environment*, *absolute consensus cannot be reached through a finite number of message exchanges*.

Strictly speaking, the Two Generals' Problem cannot be completely bypassed in theory. However, in practical engineering, its limitations can be alleviated or circumvented by the following methods, making the system feasible in real-world scenarios:

- **Improving Odds Through Redundancy:** This approach involves sending multiple messengers simultaneously. If the receiver receives some of them, it considers action to be possible. For example, in the Two Generals' Problem, instead of sending only one messenger, either side sends 100 messengers at the same time; if the receiver receives 10 of the messages (a 1/10 probability), it considers the message received. However, from an engineering perspective, this approach is not performance-friendly.
- **Introducing timeouts and default fallback actions:** The receiver can set a default time by which to expect a message. If no message from the other side arrives by that time, it takes a default safe action. For example, in the Two Generals' Problem, the army that is to begin

the attack agrees to attack at 9:00 a.m., but if no attack signal is received by 9:10 a.m., it defaults to calling off the attack. Although this default behavior cannot capture the city, it avoids the risk of failure from a unilateral attack. In distributed-system design, many designs also adopt default behavior after a timeout. For example, in the Raft algorithm, if a Follower node does not receive a message from the Leader node before the election timeout, it assumes by default that the Leader node has failed and initiates a new round of election.

Consider the real-world scenario in which two computers need to communicate. In this case, the challenge of communication is still the unreliable communication channel, so this is also one of the real-world scenarios of the Two Generals' Problem. The *TCP (Transmission Control Protocol)*² protocol can be regarded as a partial engineering solution to the Two Generals' Problem.

This book does not attempt to provide a comprehensive explanation of TCP's internal mechanics; interested readers are encouraged to refer to [4]. Here we focus on how TCP communicates over an unreliable channel to partially solve the Two Generals' Problem.

In the TCP protocol, each packet has a unique *sequence number* corresponding to it. After receiving a packet, the receiver confirms receipt by sending an acknowledgment (ACK). Through this mechanism, TCP solves the problem of packets arriving out of order. Consider the following scenario: if the sender sends packets with sequence numbers 4 and 5 to the receiver, but the receiver receives the packet with sequence number 5 first, delivering the packet with seq=5 to the application layer immediately would result in out-of-order data processing. To avoid delivering data out of order, the kernel first checks whether all packets before this packet have already been delivered to the application layer; only if this packet is the earliest undelivered packet will it be delivered. In this scenario, the packet with seq=4 has not yet been received, so the kernel first caches the packet with seq=5 and waits for the packet with seq=4 to arrive before delivering both together.



In concrete implementations of the TCP protocol stack, the “cache out-of-order data” behavior described above is not performed indefinitely; it is strictly limited by the *receive window* (rwnd).

The TCP receiver maintains a sliding-window structure. When an out-of-order packet (such as seq=5) is received, the kernel checks whether the packet's sequence number falls

²https://en.wikipedia.org/wiki/Transmission_Control_Protocol

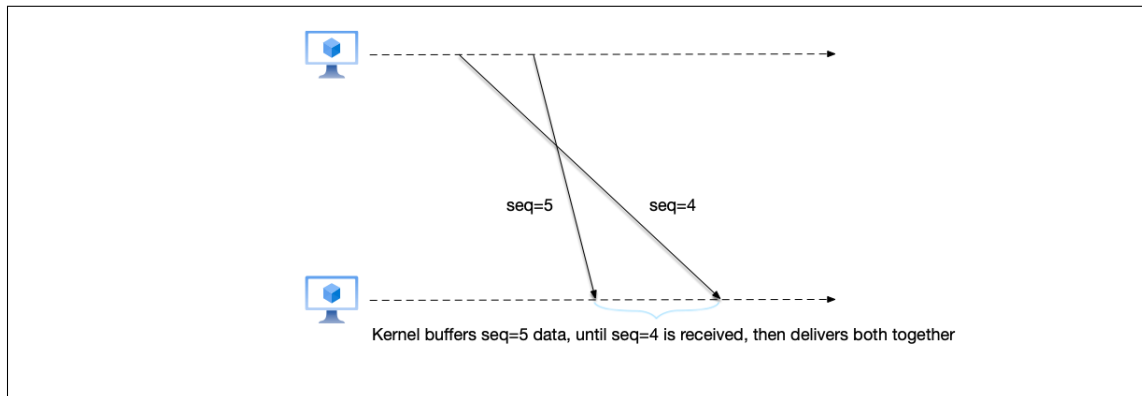


Figure 2.4: TCP uses sequence numbers to solve the out-of-order problem. The receiver first receives the packet with seq=5. Because the packet with seq=4 before it has not yet been received, the receiver must cache the packet with seq=5 until the packet with seq=4 arrives, and then deliver both packets to the application layer together.

within the currently allowed receive window:

- If it is outside the window range: the packet is dropped directly to prevent the receiver's memory from being exhausted.
- If it is out of order because a previous packet (such as seq=4) is missing: the TCP protocol stack temporarily stores seq=5 in an out-of-order queue and sends a duplicate ACK to the sender for the data before seq=4, triggering the sender's fast-retransmit mechanism.
- Only when the missing seq=4 finally arrives does TCP splice seq=4 with seq=5 from the out-of-order queue into a continuous byte stream, advance the left boundary of the sliding window, and finally submit it to the application layer. This mechanism reflects an important trade-off in distributed-system design: by consuming receiver memory, network-transmission bandwidth is saved (avoiding retransmission of already-arrived out-of-order packets).

Sequence numbers solve the out-of-order delivery problem, while data loss is solved by the timeout-retransmission mechanism. In the TCP protocol, after sending a packet, the sender starts a retransmission timeout for that packet. If the sender has not received the receiver's ACK response within the specified time, it considers the packet lost and retransmits it. TCP will make a best-effort attempt to retransmit multiple times; if all retries fail, it aborts the connection and returns an error.

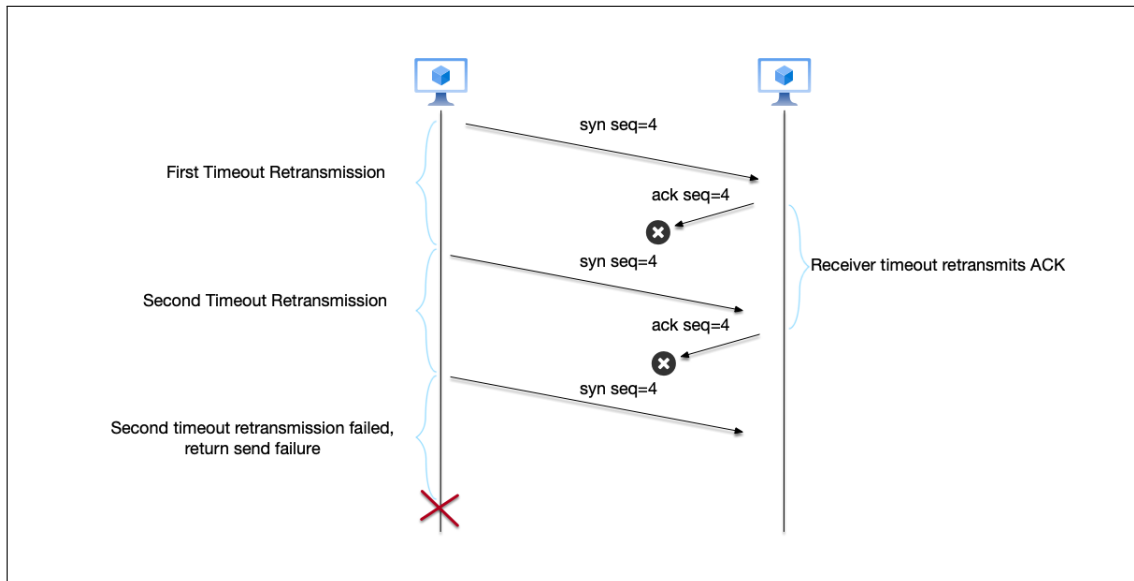


Figure 2.5: TCP uses timeout retransmission to solve data loss. Note that in the figure, both the sender sending data and the receiver acknowledging receipt rely on timeout retransmission to ensure delivery.

As we can see, TCP significantly improves communication reliability through sequence numbers for out-of-order delivery and through retransmission to ensure every packet is successfully sent. However, TCP still cannot completely solve the Two Generals' Problem; it is at best a partial engineering solution. For example, if someone unplugs the network cable, no matter how many retransmissions occur, the data certainly cannot be sent or received successfully.



In Figure 2.5 we can see that the receiver also needs timeout retransmission when sending ACK responses, which means the sender also needs an acknowledgment for the ACK response. The explanation of TCP above omits many details for simplicity; interested readers may refer to the books recommended earlier.

Also, in Figure 2.5, considering two retransmission failures as a transmission failure is also a simplification.

2.2 Byzantine Generals' Problem

The *Byzantine Generals' Problem*³ was first proposed by Lamport in the paper [7] and is likewise a thought experiment about distributed systems. The Two Generals' Problem discusses problems in distributed systems caused by network communication; the Byzantine Generals' Problem, by contrast, discusses problems caused by the nodes themselves.

Similar to the Two Generals' Problem, the Byzantine Generals' Problem also assumes a scenario in which armies coordinate an attack on a city. In the Byzantine Generals' Problem, the armies' actions include not only attack but also retreat; a mix of armies attacking and retreating may also lead to failure. In addition, unlike the Two Generals' Problem:

- In the Byzantine Generals' Problem, more than two armies may participate in the attack.
- In the Byzantine Generals' Problem, it is assumed that messages communicated between armies are always received correctly; there is no situation, as in the Two Generals' Problem, where messengers are captured and messages fail to arrive. That is, in the Byzantine Generals' Problem, communication is assumed to be reliable.

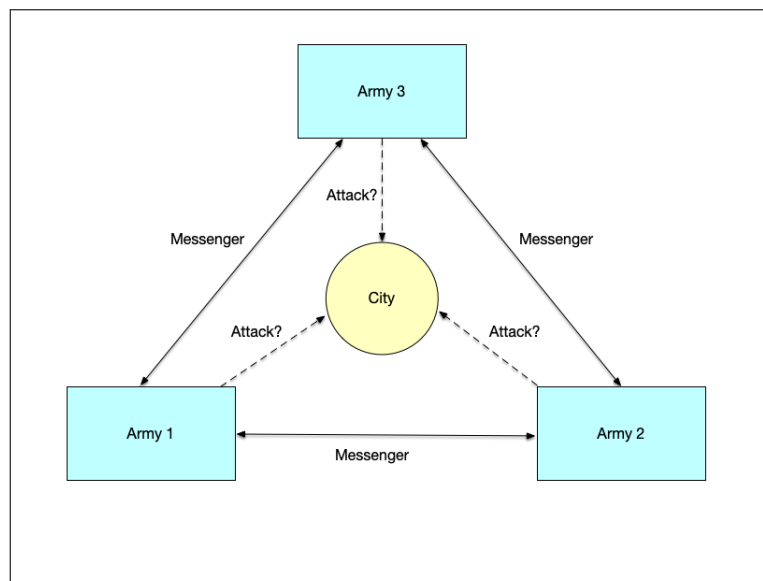


Figure 2.6: The Byzantine Generals' Problem

While the Byzantine Generals' Problem assumes reliable communication, unlike the Two Generals' Problem, it introduces a new complication: traitors may exist among the generals. These traitors may intercept and alter messages or deliberately disseminate false information.

³https://en.wikipedia.org/wiki/Byzantine_fault

Figure 2.7 gives two examples of what happens when traitors appear. In the figure, normal messages are shown with solid lines, while malicious messages sent by traitors are shown with dashed lines. In the upper part of the figure, after General 2 receives General 1's attack message, he passes a tampered, malicious message to General 3: "General 1 says retreat." General 3 then receives the attack message sent by General 1. These two contradictory messages leave General 3 confused: who is lying? In this scenario, the traitor node maliciously alters the content of the message when forwarding it.

Likewise, in the lower part of the figure, the traitor becomes General 1, who says "retreat" to General 2 and "attack" to General 3. In this scenario, the traitor node maliciously sends misleading messages to different nodes.

Although these are two different scenarios, from General 3's point of view, he sees two contradictory messages.

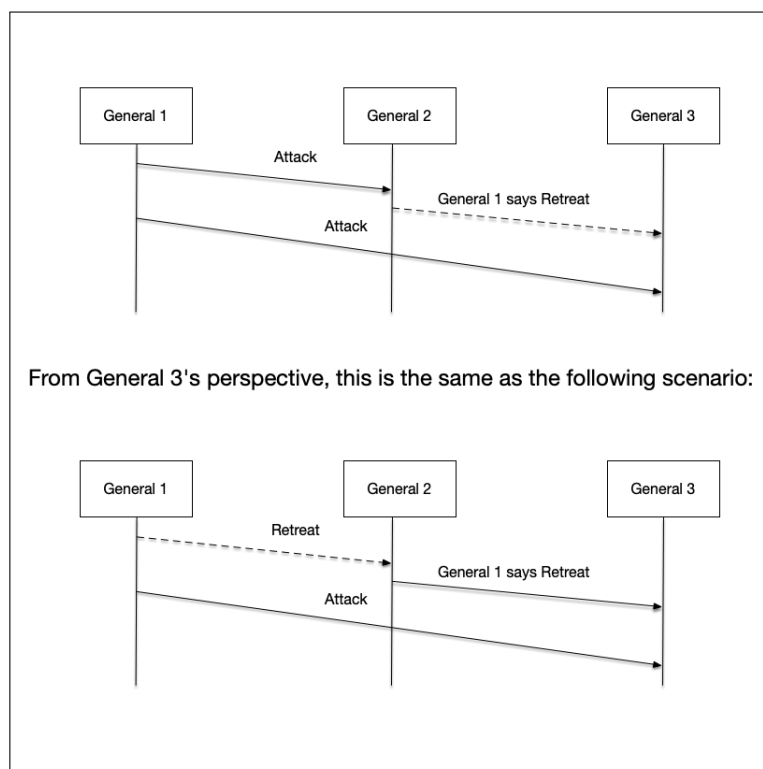


Figure 2.7: A traitor appears in the Byzantine Generals' Problem

Mapping the Byzantine Generals' Problem to a distributed system, the generals are the nodes, the messengers are the communication links between nodes, and the Byzantine-failure model describes a situation in which some malicious nodes in a distributed system may tamper with or send misleading messages, causing system failures.

A paper by Cynthia Dwork et al. [8] proves that in a system with malicious nodes and unpredictable communication delays, the Byzantine Generals' Problem can be solved only if strictly fewer than one-third of the nodes are malicious. That is, in a system with $3f + 1$ nodes, no more than f nodes may be malicious. For example, in a system with four nodes, at most one malicious node can be tolerated.

In some distributed systems, it is necessary to consider that malicious nodes may exist in the system. Such systems are called *Byzantine fault tolerant*, for example in the blockchain and cryptocurrency domains. These systems must provide guarantees that they can operate normally even when malicious nodes exist. We will continue to discuss this topic later in the chapter.

2.3 System Models

The first two sections of this chapter discussed two thought experiments in distributed systems:

- Two Generals' Problem: demonstrates failures that may be caused by *network communication*.
- Byzantine Generals' Problem: demonstrates failures that may be caused by *node behavior*.

Both thought experiments are too “idealized” in that they assume only one type of failure. In real distributed systems, multiple types of failures usually coexist. Before designing a distributed system, we must first clarify the environment in which the system operates: what types of failures must be tolerated, and what consistency requirements must be satisfied. For example, the Paxos and Raft algorithms described later in the book are consensus algorithms that do not work in Byzantine-failure environments.

Models of distributed systems are theoretical frameworks and abstractions for describing and analyzing distributed-system behavior, aiming to help understand, design, and optimize distributed systems. They mainly include the following:

- Communication model: describes how nodes exchange information.
- Failure model: describes how nodes may fail.
- Timing model: defines how confident we can be about “delay” when designing algorithms.
- Consistency model: defines what consistency requirements the different nodes of a distributed system must satisfy for their data.

This chapter discusses the first three models; the consistency model is discussed in Section ??.

2.3.1 Communication Models

“The network is reliable” is the first of the eight fallacies of distributed computing. The article “The Network is Reliable”[9] lists various causes of network failures, including real-world examples: network maintenance, router failures, power failures, and even accidentally unplugging a power cord or cutting a fiber-optic cable.

When discussing network models, we take a more abstract perspective and ignore the details of these failures. Most distributed algorithms assume that nodes provide bidirectional message passing, known as *point-to-point* or *unicast communication*.

In general, the following link-reliability guarantees exist: *Fair-Loss Link*, *Reliable Link*, and *Authenticated Link*.

Fair-Loss Link

In a fair-loss link, messages may be lost, and some messages may be reordered or duplicated, but fair-loss links have the following core properties:

- Finite loss: messages may be lost, but not indefinitely. If a message is sent infinitely many times, it will eventually be delivered. That is, as long as the physical link is not broken, the probability of success is greater than zero.
- No spurious messages: if a receiver receives message m , the sender must have actually sent it (assuming no Byzantine-style malicious forgery).

The fair-loss link assumes that messages will eventually be delivered after repeated attempts. This assumption implies that network partitions will last only a finite amount of time, not forever. Therefore, as long as the sender does not crash, it can always guarantee eventual delivery through infinite retries; however, if the sender crashes before the message is delivered, it cannot retry. But it has many problems: packet loss is the norm, messages may be duplicated and reordered. It can serve as the foundation of all higher-level links, but business logic cannot be directly built on it.

Reliable Link

A reliable link is also called a *perfect link*. It has the following properties:

- Adds a message-retransmission mechanism to ensure messages are eventually delivered.
- Adds a mechanism to detect and suppress duplicate messages.

A reliable link guarantees reliable delivery of every message sent: if a process sends a message, it will eventually be delivered correctly. A reliable link can also guarantee that no message is delivered more than once. In short, reliable delivery and no duplicate messages mean that every message sent by a correct process is delivered exactly once.

The reliable link is precisely the communication model that the TCP protocol attempts to provide, and it is the environment usually assumed by algorithms such as Raft and Paxos. It is the standard abstraction we pursue in distributed systems.

Authenticated Link

When a system runs in an untrusted environment (such as the public Internet or blockchain) and faces Byzantine failures, “reliability” alone is not enough; “security” is also required. The challenges include:

- Man-in-the-middle attacks: an attacker intercepts a sent message, modifies its content, and sends it to the receiver.
- Data forgery: an attacker impersonates the sender and sends messages to the receiver.

Authenticated links were proposed to solve these problems. Their properties are:

- Integrity: the message has not been tampered with during transmission.
- Authenticity: the receiver can be sure the message really came from the sender.

The common SSL protocol implements authenticated links by establishing encrypted channels.

Among these network links, weaker links can be transformed into stronger links by adding technical means on top of the model, as shown in Figure 2.8:

- By repeatedly retransmitting lost messages until they are delivered and filtering out duplicate messages at the receiver, a fair-loss link can be transformed into a reliable link.
- Using encryption technology, any link can be transformed into a fair-loss link. For example, the HTTPS protocol adds TLS encryption to HTTP (the “S” in HTTPS) to prevent attackers from eavesdropping on or tampering with messages. However, encryption cannot prevent messages from being dropped by the receiver; therefore, only when we assume the receiver will never block message communication can any link be transformed into a fair-loss link.



Although the TCP protocol attempts to provide reliable-link guarantees, when designing distributed systems, do not naively assume that TCP is a “reliable link.” While TCP provides the abstraction of a reliable link while the connection is alive, once the connection is reset or the machine is rebooted, the TCP context is lost. Therefore, a true distributed system must implement a “reliable link” at the application layer:

- The application layer has its own request IDs for deduplication.

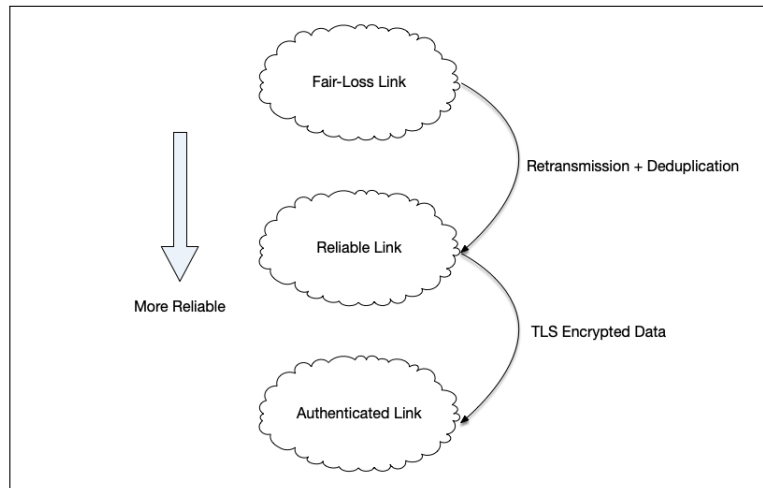


Figure 2.8: Transforming weaker network links into more reliable links through technical means

- The application layer has its own ACK mechanism to confirm successful business processing, not just TCP packet receipt.
- The application layer has its own retry strategy.

This is why RPCs in the Raft paper include Term and Index: these are the tools the application layer uses to build truly reliable links that span TCP connection lifetimes.

2.3.2 Failure Models

In real systems, node and network failures may occur simultaneously. As shown in Figure 2.9, Node A sends a request to Node B asking for the value of variable x . After receiving the request, Node B replies to Node A that the value of x is 5.

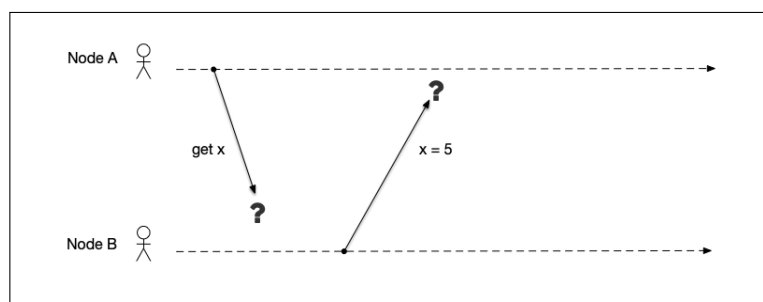


Figure 2.9: Possible failures during communication between two nodes

The following failures may occur during this simple communication process:

- Node B does not receive Node A's request because of a communication failure.
- Node B crashes while processing the message and cannot reply.
- Node B receives the request only after a long delay because of network latency.
- Node B is too busy (for example, processing other transactions) to respond to the message.
- Node B maliciously replies with incorrect data to Node A.
- The message is lost during transmission or reception.
- When Node B replies, Node A crashes and cannot receive the reply.
- Etc.

It is clear that even in this simplest scenario, it is impossible to enumerate all possible failures. However, failures can be classified; different failure models have different design difficulties, and systems must be designed based on assumptions about a particular failure type.

Failure models define how badly machines, networks, and software can fail in the working environment of a distributed system. In the theory and practice of distributed systems, failure models form a hierarchy. From simple “crash” to complex malicious “tampering”, the difficulty of handling these failures increases exponentially.

2.3.2.1 Crash Model

In a *crash fault*, a node may crash for various reasons, such as hardware failures or software bugs. Depending on whether a node can resume execution after a crash, crash faults are further divided into *crash-stop* faults and *crash-recover* faults.

Crash-Stop Model

In a *crash-stop* fault, it is assumed that after a node crashes, it never resumes execution. This is the most idealized model and usually exists only in theoretical derivations or extremely controlled hardware environments. Under this failure model, once a node fails, it immediately stops all operations; other live nodes in the system can accurately detect that the node has died. Algorithms for handling this failure model are very simple; for example, primary-backup failover is used. Once the primary node is detected to be down, the backup node immediately takes over.

This type of failure applies to unrecoverable hardware errors and some *stateless* services.

For example, certain simple layered Web services consist of a front-end API service and a back-end database service. If traffic increases, one scaling solution is to add additional front-end API services behind a load balancer. By adding API servers, more user requests can be accepted. In the design of such systems, a principle followed is to make the front-end services *stateless*. As

the name implies, “stateless” means that when creating a process instance, no past data or state is stored, and no data needs to be persisted. Using stateless API services means that each individual API service does not track any user information outside the context of a single request. One advantage of stateless services is that they can be stopped at any time and started freely; no attempt is made to recover state upon startup, which facilitates seamless system scaling.

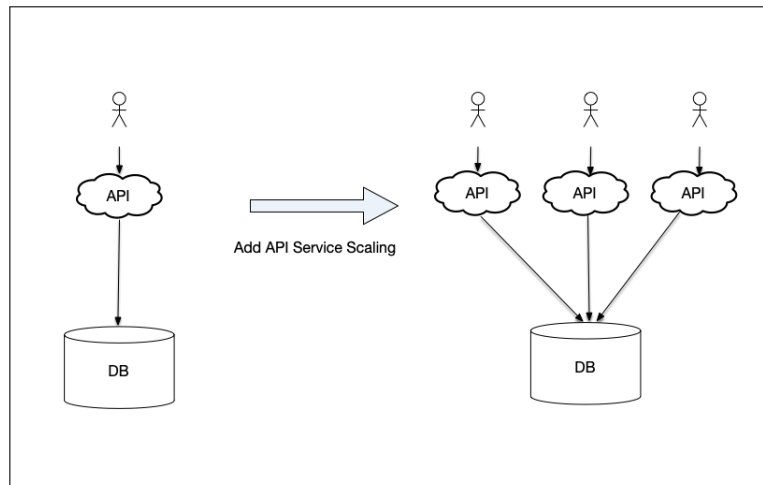


Figure 2.10: Increasing request-handling capacity by adding stateless API services

Another example is stateless services in Kubernetes. In Kubernetes’ stateless-node pattern, a service consists of multiple Pods. Each Pod runs only once during its lifecycle; once a Pod is restarted, it may be scheduled to run on another node. Each time a Pod is created, it is assigned a unique ID. If a node goes down, Pods scheduled to that node will be deleted after a timeout. If Pods are configured to restart after a failure, they will discard all data in memory when the failure occurs, and will not attempt to recover the state before the crash when restarted.

Crash-Recover Model

A *crash-recover* fault means that after a node crashes, it continues execution by restarting. In this failure model, we assume nodes may crash but can always recover within a certain time. If it is necessary to resume execution from the state before the crash, this can be done by reading persisted data. This is the core assumption model for distributed storage systems in industry. Under this model, the following phenomena may occur:

- Nodes may crash at any time and stop responding.
- Nodes may restart after a period of time and rejoin the cluster.
- A node’s “memory” may be lost: *volatile state* in memory, such as locks, caches, and uncommitted transactions, will all be lost.

Therefore, the core challenge for such systems is how a node obtains the latest state of the cluster after recovering from a crash:

- After restarting, the node's state is stale. It must know how far behind it is and "catch up" from other nodes.
- If a node was a Leader node, after it crashes the cluster elects a new Leader. A moment later the old Leader restarts, still thinking it is the Leader and trying to give orders.

Database services are the most typical example of crash-recover fault services. When a process fails, the database loads its previous state by loading snapshot data before the crash and replaying the write-ahead log (WAL)⁴, among other means, to recover the data from before the crash.

In Kubernetes, developers can use the *StatefulSet* type to create applications. Pods created in this way provide uniqueness guarantees that are preserved even after restarts and rescheduling. When an individual Pod fails, the Pod identifier can be used to read data from the existing data volume to recover state.

2.3.2.2 Omission Model

An *omission fault* means that under certain circumstances a node omits sending or receiving some messages. Causes of omission faults usually include packet loss due to network instability and exhaustion of system resources.

This model is more subtle than the crash model. The node is still running, but it cannot send or receive messages normally. Its core challenges are:

- Ineffective failure detection: the node's heartbeat packets may fail to go out, causing the node to be misjudged as "dead." But in fact it is still writing to disk and still processing local logic. This can easily cause data inconsistency.
- Asymmetric network: Node A can send messages to Node B, but Node B cannot send messages to Node A.

To understand omission faults more intuitively, we can dive into the operating-system kernel and network-hardware layers. In engineering practice, omission faults often manifest as a "silent drop," which is harder to debug than an explicit error:

- Receiver omission: when a node is under extremely high load (for example, CPU at 100% capacity), although the OS kernel is still running, it may not be able to process network-card interrupts in time. At this point, the network card's hardware buffer (ring buffer) or

⁴https://en.wikipedia.org/wiki/Write-ahead_logging

the kernel's receive queue fills up quickly. Once it overflows, the network card silently drops subsequently arriving packets at the hardware level. In this case, the application layer is completely unaware that messages have arrived, while from the sender's point of view it is as if the messages have sunk into the ocean. At this point the node is still processing local disk I/O, but in the cluster network it has become a "black hole."

- Sender omission: at the level of intermediate devices such as switches or routers, if transient micro-bursts occur that exhaust the switch port's buffer, packets are also dropped by intermediate devices. Node A successfully calls the `send()` interface and gets a successful return value (because the data was written to the local socket buffer), but the packet never actually reaches Node B.

Common technical solutions include:

- Retry and acknowledgment: every message must be confirmed by the receiver before it is considered successfully delivered. If no acknowledgment is received for a long time, the message is retransmitted, turning "omission" into "delay."
- Lease: to perform a write operation, a valid lease must be held. If an omission fault prevents the lease from being renewed, the lease expires and the node is forced to stop serving (even if it thinks it is still alive).

2.3.2.3 Byzantine Model

In the *Byzantine fault* model, a node may perform arbitrary actions, including crashes or malicious behavior. When designing algorithms for this model, no assumptions can be made about the behavior during a failure.

This model is the most general and also the hardest to design algorithms for. In terms of performance and storage cost, it is the most expensive. However, when unknown or unpredictable failures may occur, it is the only reasonable choice.

Note that the "arbitrary failures" mentioned here are not necessarily malicious or deliberate; they may also be caused by incomplete implementation, framework or system bugs, or hardware failures.

For example, in a decentralized network such as Bitcoin[10], there may be participants trying to cheat for profit. In such a system, trusting the messages or data of other nodes is unsafe. Bitcoin avoids this problem through proof of work (PoW)⁵.

In the article "A Byzantine Failure in the Real World"⁶, the Cloudflare team mentions a

⁵https://en.wikipedia.org/wiki/Proof_of_work

⁶<https://blog.cloudflare.com/a-byzantine-failure-in-the-real-world/>

production incident in 2020 caused by an abnormal switch.

Failure Hierarchy

Failure models have the hierarchical relationship shown in Figure 2.11. The innermost crash-stop model is the easiest to handle; the outermost Byzantine model is the hardest. If a system can handle the harder failure types in the outer layer, it can naturally handle those in the inner layer. For example, a system that can handle the crash-recover model can certainly handle the crash-stop model. We explain them from the inside out.

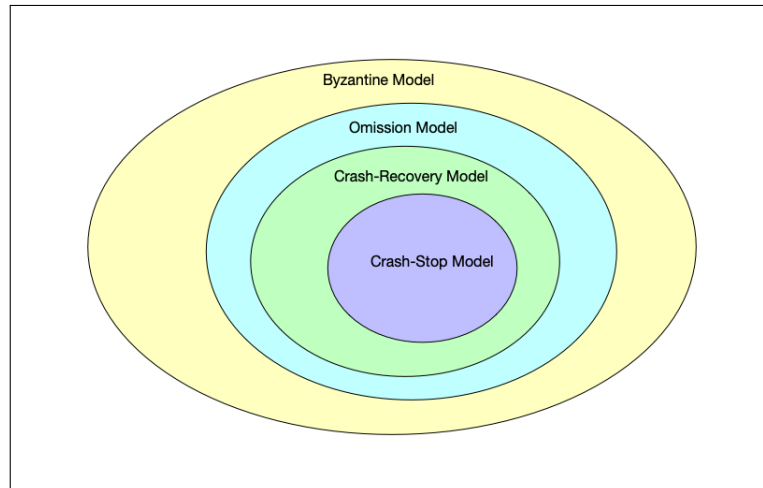


Figure 2.11: Failure hierarchy

Crash-Stop Model

In the crash-stop model, after a node crashes it stops running and does not recover. Therefore, its set of failure behaviors is very small: only $\{Stop\}$.

Crash-Recover Model

In the crash-recover model, a node can recover and resume running after a crash. Therefore, the crash-stop model can be viewed as a special case where the recovery time is infinite. Its set of failure behaviors is $\{Stop, Restart\}$.

Omission Model

In the omission model, a node may drop sent messages or received messages. From this characteristic, the crash model is essentially an extreme case of the omission model. When a node crashes, it no longer sends messages or processes received messages. To an external observer, this is 100% send omission and 100% receive omission. In addition, the omission model allows nodes to be in certain intermediate states: for example, the network card is broken, so it can receive messages but cannot send them; or the CPU is busy and drops some packets.

In the crash model, once a node stops responding to messages, it can be safely assumed

to be down or restarting. However, in the omission model, the same lack of response might simply mean that packets are being lost, even though the node is still alive and writing to disk. If the system incorrectly assumes the node is down and elects a new primary, it can lead to a split-brain scenario.

In summary, the set of omission-failure behaviors is $\{Stop, Restart, Drop_Send, Drop_Receive\}$.

Byzantine Model

The behavior pattern of the Byzantine model is that a node can send arbitrary data, including lies, forgeries, and contradictory messages. From this perspective, the omission model is just a special case: a Byzantine node can choose to “pretend not to have received a message” (simulating receive omission), or “pretend it cannot send a message” (simulating send omission), or “I simply crash” (simulating a crash). In addition, a Byzantine node can do things the omission model cannot: it does not drop messages, but it can tamper with them, for example changing “Agree” in a message to “Reject” before sending it to another node.

Although the omission model may drop packets, as long as a packet is received its content can be trusted; but in the Byzantine model, even if a node receives a packet it cannot trust it—signatures and tamper-proofing are required.

In summary, the set of Byzantine-failure behaviors is $\{Stop, Restart, Drop, Lie, Forge, \dots\}$, the union of all possible failures.

Engineers need to understand different failure models and their compatibility relationships to ensure that the designed system can correctly handle the failures that may occur in its environment, while also avoiding over-engineering, because the design difficulty varies greatly across failure models. For example, a system deployed in a trusted internal network does not need to be designed to satisfy the Byzantine-failure model; conversely, if a system is exposed to the public Internet where arbitrary nodes may join, it should be designed to satisfy the Byzantine-failure model.

2.3.3 Timing Models

The assumed time bounds and communication delays of a system largely determine the algorithm used to solve the problem in the end. Timing models are generally divided into three types: *synchronous*, *asynchronous*, and *partially synchronous*. These three models determine how confident we can be about “delay”; this is the watershed between theory (e.g., the FLP impossibility result) and industry (e.g., Raft/Paxos).

Synchronous Model

The synchronous model is the most idealized and simplest, yet it is virtually nonexistent on the public Internet. It means that in a system, three physical properties have strict *upper bounds*:

- Bounded message-transmission delay: when a node sends a message, it is guaranteed to arrive within a known time Δ . If Δ time passes without arrival, the system can conclude with 100% certainty that the network is broken or the other side has crashed.
- Bounded processor speed: the time for each machine to execute one step is predictable and will not be infinitely slow.
- Bounded clock drift: the clock discrepancy between any two machines will not exceed ϵ .

These assumptions mean that failure detection for such a system is very simple.

Asynchronous Model

The asynchronous model represents the polar opposite of the synchronous model. It means that in a system, there are no limits on the three physical properties:

- Message delay has no upper bound: a message may wander in the network for 1 millisecond or 100 years; as long as it eventually arrives, it satisfies the model.
- Processing speed has no lower bound: a machine may suddenly be suspended by the operating system or enter a long GC pause, sleeping for several minutes before waking up.

In the asynchronous model, these assumptions imply:

- It is impossible to distinguish “slow” from “dead”: if Node A sends a message to Node B and gets no response, you never know whether Node B has crashed, the network is extremely slow, or Node B is processing complex computation.
- FLP impossibility: in an asynchronous system, as long as one node may crash, there is no deterministic consensus algorithm that can guarantee agreement is always reached.

Partially Synchronous Model

Both the synchronous and asynchronous models are too extreme; in most cases, systems operate under the partially synchronous model, which is the main battlefield of industry. Its core characteristic is that the system behaves like a synchronous system most of the time, but behaves like an asynchronous system during unpredictable, finite periods.

There are many reasons why a system may violate synchronous assumptions. We list some common ones below:

- Messages need to be retransmitted after being lost, especially in the case of network partitions, where the upper bound of delay may be infinite.
- Network congestion also causes message delays, because packets queue in switch buffers.

- Network reconfiguration may also cause large delays. In a 2012 incident at GitHub⁷, packets within a single data center were delayed by more than one minute.
- When executing algorithms on a node, the expected speed is constant, because each instruction has a fixed number of CPU clock cycles. But there are also factors that may cause a running program to pause for a long time. Some possible reasons are:
 - The OS kernel scheduler may preempt some running processes, pausing their execution to let other processes run.
 - In garbage-collected languages such as Java, system pauses caused by garbage-collector runs (stop-the-world, STW).
 - When remaining memory is insufficient, page faults and memory swapping that may occur when a program requests memory from the kernel can cause a process to be suspended.

2.4 Chapter Summary

This chapter serves as the theoretical foundation of the book. It focuses on how to abstract and model complex distributed-system environments. We established that, unlike single-node systems, distributed systems lack shared memory and global clocks, and they must confront inherent uncertainties in network and node behavior. To meet these challenges, we use thought experiments and theoretical models to define the boundary conditions that must be considered when designing systems.

This chapter first uses the two classic generals' problems to reveal the two most fundamental difficulties in distributed systems:

- Two Generals' Problem (uncertainty of the network): demonstrates the "infinite acknowledgment" trap encountered when trying to reach consensus over an unreliable channel. It proves that in an unreliable communication environment, absolute consensus cannot be reached through a finite number of message exchanges. Although the TCP protocol alleviates this problem through sequence numbers and retransmission mechanisms, it is only a partial engineering solution, not a complete theoretical solution.
- Byzantine Generals' Problem (untrustworthiness of nodes): demonstrates the challenges when malicious nodes (traitors) in the system tamper with or forge messages. Theory

⁷<https://github.com/blog/1364-downtime-last-saturday>

proves that in a system containing malicious nodes, consensus is possible only when the number of malicious nodes is less than one-third.

To analyze system behavior more precisely, we define three core models that together form the environmental assumptions for distributed-algorithm design:

- **Communication model:** we introduced fair-loss links, reliable links, and authenticated links. Although TCP provides reliable transport while a connection is alive, true distributed “reliable links” must be implemented at the application layer, including request-ID deduplication, application-layer ACKs, and retry strategies, to cope with connection resets and context loss after machine restarts.
- **Failure model:** we built a failure hierarchy from simple to complex.
 - **Crash-stop:** the simplest model, applicable to stateless services.
 - **Crash-recover:** the most common model in industry (e.g., databases, Kubernetes), where nodes may restart and face state loss and catch-up problems.
 - **Omission faults:** nodes may drop received messages or fail to transmit them, making failure detection notoriously difficult.
 - **Byzantine fault:** the most complex failure model, where nodes may arbitrarily lie or behave maliciously, encompassing all the failure types above.
- **Timing model:** the timing model determines the basis on which algorithms judge “time-out.”
 - **Synchronous model:** strict upper bounds on delay, speed, and clock drift; extremely rare in practice.
 - **Asynchronous model:** no upper bound on message delay, making it impossible to distinguish a “slow” node from a “dead” one.
 - **Partially synchronous model:** the main battlefield of industry, assuming the system is synchronous most of the time but allowing brief asynchronous periods (such as network congestion or STW caused by GC).

Through the study of this chapter, we have not only mastered the theoretical terminology but, more importantly, understood the trade-offs between models. When designing a system, engineers must be clear about the environmental model they are in: are they handling crash-recovery in a trusted internal network (e.g., Raft), or defending against Byzantine attacks in an open network (e.g., blockchain)?

Time and Order in Distributed Systems

In distributed systems, multiple nodes work together. Client requests are sent to different nodes for processing, and these requests become events on individual nodes. As we will see, the system's state is formed by executing these events one by one in a *specific order*. Therefore, the order of events is particularly important. Different nodes may see different sequences of events, which can lead to different states on these nodes.

From this, it is evident that when processing multiple events, the order in which they occur affects the state of nodes. Thus, how to measure the order of events becomes a core problem in distributed systems. A natural idea is to sort events according to their *physical time*. In Section 3.2, we will learn about the principles of physical clocks and, unfortunately, discover that in a distributed system, comparing physical time across multiple nodes to determine the order of events is not precise and can sometimes even lead to errors. Once the order of events is disrupted, the state of the entire distributed cluster becomes corrupted.

If physical time is not feasible, what other method can be used to determine the order of events in a distributed system? The answer is *logical clocks*.

Surprisingly, although we have repeatedly emphasized that the order of events is particularly important, in distributed systems there may still be cases where the order of occurrence of two events cannot be determined. Such events are called "*concurrent events*." To explain event ordering, we also need to understand two mathematical definitions: *partial order* and *total order*. In fact, these two mathematical definitions should be deeply rooted in the mind of every distributed systems engineer, and we will encounter their applications in distributed systems

again and again in later chapters.

In this chapter, we will discuss the following topics:

- How the order in which events occur affects the state of the system;
- Why physical time cannot be used to measure the order of events in distributed systems;
- The theoretical foundation and computation rules of logical clocks;
- Intuitive explanations and definitions of partial order and total order;
- Vector clocks, an extension based on logical clocks.

3.1 State, Events, and Snapshots

In some crime dramas, we often see scenes like this: police are tracking a suspect who suddenly changes their behavior one day, so the police pull up records to review where the suspect went, who they met, and what they did over the past day, in order to understand what caused the suspect's change.

Here, “places visited, people met, and things done” are individual *events*. If we view the suspect as a *system*, the suspect from yesterday and the suspect after these events represent the system's states at different points in time. (From this, we might ponder a philosophical question, as shown in Figure 3.1: the “me” of the past, after going through a series of events, becomes the “me” of the present—are these two “me’s” the same “me”?)

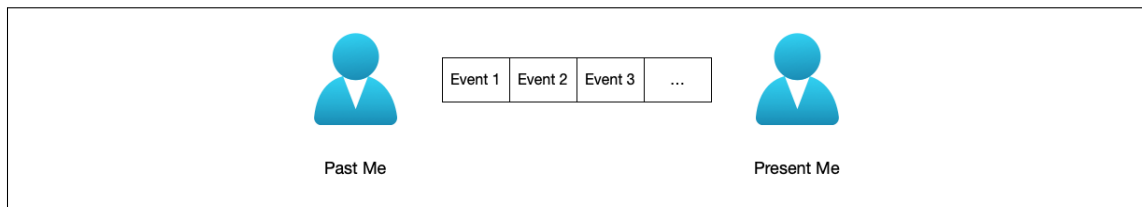


Figure 3.1: The “me” of the past becomes the “me” of the present after a series of events—are these two “me’s” the same “me”?

Let us continue explaining state and events with a simple storage service. As shown in Figure 3.2, a storage service initially has state $\{x = 1\}$. After sequentially executing commands $set\ x = 2$ and $set\ x = 3$, the new state becomes $\{x = 3\}$. Here, the execution order of these two commands is particularly important; if the order of these two events is reversed, the resulting state becomes $\{x = 2\}$.

From the examples above, we can also see that we have not mentioned the *physical time* of event execution, but rather focus more on the *order* of event execution. The reason for focusing

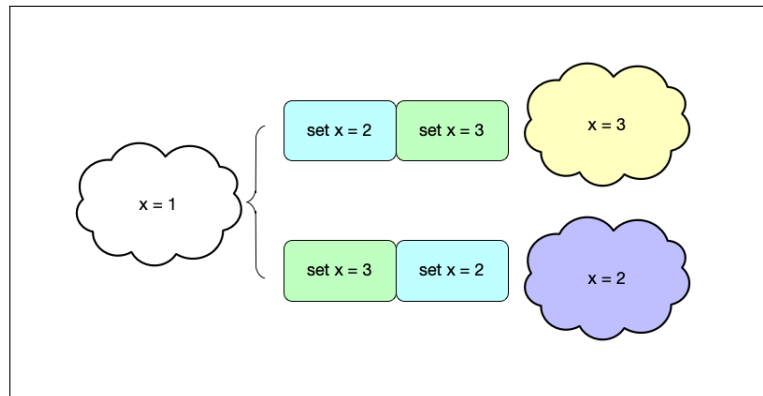


Figure 3.2: With the same initial state, executing the same events in different orders produces different states

on the relative order between events rather than physical time is that there is no globally unified physical time in a distributed system; we will continue to discuss physical time in depth in Section 3.2. If we view the system as a large state machine, events are the operations that change the state of this state machine. In a state machine, as long as we can guarantee that events are executed in the *same order* every time, we can ensure that the system always reaches the same state.

Executing the same events in the same order produces the same result—this is the core idea of *state machine replication*¹:

If two different processes start from the same initial state and process input data in the same order, they will produce the same output.



After discussing logical clocks in Section 3.6, we will see that the *execution order* of events is, in a sense, *logical time*.

On the other hand, the instantaneous state of this storage at different points in time is called a *snapshot* at that point in time. For example, taking a photograph on a busy street captures a snapshot of that street at that exact moment. In contrast, events are *dynamic* data used to change the system state, while snapshots are *static* data representing the system state.

At this point, we can give intuitive explanations of state, events, and snapshots:

¹https://en.wikipedia.org/wiki/State_machine_replication

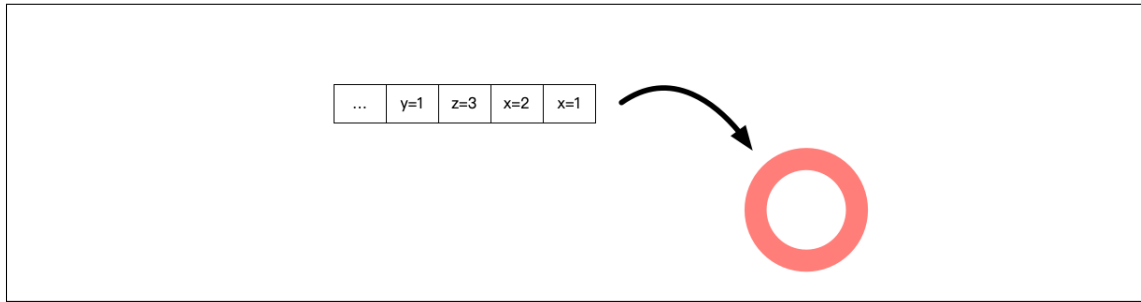


Figure 3.3: Viewing the system as a large state machine, events are the operations that change the state of this state machine. In a state machine, as long as we can guarantee that events are executed in the *same order* every time, we can ensure that the system always reaches the same state.

- State: all data values in a system; state changes with events;
- Event: an operation that can change the system state. In a service, all write requests (writes, updates, deletions, etc.) can be called events;
- Snapshot: a snapshot is a special kind of state; it is a *static* state. Once the time of the snapshot is determined, its content is also determined.

As shown in the figure below, if we obtain snapshots of the system at the initial time, after executing command `set x = 2`, and after executing command `set x = 3`, we will obtain $\{x = 1\}$, $\{x = 2\}$, and $\{x = 3\}$ respectively.

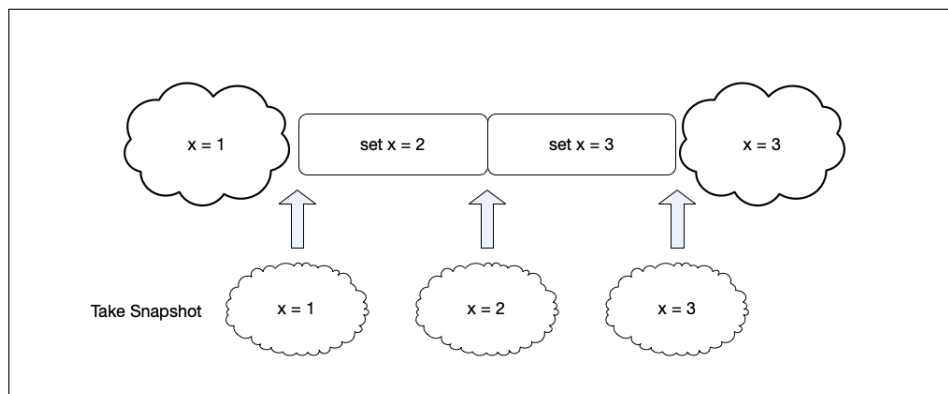


Figure 3.4: A snapshot is state data at an instantaneous moment

From the above explanations of state, events, and snapshots, we can see that these concepts are all strongly related to time, especially the relationship between events and temporal order. Next, we will look at how to measure the order of events on different nodes in a distributed system.



Dividing the data related to a system into three categories—state, events, and snapshots—is also the approach used in data replication in distributed systems. Here we emphasize that in distributed systems, maintaining the same *order* of events across multiple replicas is required; a stronger requirement than this is *determinism*, which we will continue to discuss when we talk about data replication between replicas in Section ???. We will also see in Chapter ?? that consensus algorithms are essentially log systems maintaining the same order across multiple replicas.

3.2 Physical Clocks

3.2.1 Physical Time Sources and Representation

Most computer clocks today use *quartz clocks*. Quartz clocks are usually powered by batteries, with a quartz crystal (typically in a tuning-fork shape) at their core. When voltage is applied to the quartz crystal, it generates mechanical vibrations due to the piezoelectric effect, with a very stable frequency (usually 32,768 Hz). This high-precision oscillation is the foundation of quartz clock timing. After a predefined number of oscillations, the timer chip inside the computer interrupts once (called a *clock tick*). The interrupt handler increments a counter that records the number of ticks since some past epoch. Knowing the number of ticks per second, we can calculate the year, month, day, and time.

Quartz clocks are inexpensive but not accurate enough. Due to manufacturing differences, each quartz crystal has a slightly different oscillation frequency. Additionally, quartz crystals operate at room temperature, and temperature increases or decreases change their vibration frequency. Since every computer has a quartz clock, these differences in oscillation mean that clocks in different computer hosts may drift—one node may run slow while another runs fast.

Quartz clocks may have their vibration frequency altered by environmental factors, causing the clock's running speed to change. So, is it possible to find a substance that can vibrate at a constant frequency? If so, time could be represented as the number of vibrations of this substance. In the 1930s, Isidor Isaac Rabi² and his students at Columbia University's laboratory studied the fundamental properties of atoms and atomic nuclei. They eventually discovered that the resonance frequency of certain atoms is extremely accurate and could be used to build

²https://en.wikipedia.org/wiki/Isidor_Rabi, Nobel Prize in Physics 1944

highly precise clocks. *Atomic clocks*³ are timing devices based on atomic resonance: they are built on the principles of quantum mechanics and represent the most precise timing technology humans have mastered, with an error of only one second over millions of years. This extreme precision stems from the electromagnetic wave frequency radiated when electrons inside atoms transition between specific energy levels. This frequency has extremely high stability and is almost completely immune to external environmental disturbances such as temperature and pressure.

In fact, *International Atomic Time (TAI)*⁴ is based on this principle to define the standard unit of time: 1 second is defined as the duration of 9,192,631,770 periods of the radiation corresponding to the transition between two hyperfine levels of the ground state of the cesium-133 atom. Although atomic clocks are expensive, their unparalleled accuracy has made them indispensable infrastructure for scientific research, modern communications, and global positioning systems (GPS).

Atomic clocks seem to work very well, but they ignore the astronomical concept of time. People's perception of a day is based on sunrise and sunset. In astronomical terms, a "day" refers to the time it takes for the Earth to rotate once. Due to factors such as tides, earthquakes, and glacier changes, the Earth's rotation speed is not constant. For example, on August 5, 2025, the Earth's rotation speed increased, making it the shortest day on record⁵. This day was 1.25 milliseconds shorter than 86,400 seconds.

From this perspective, on the question of "how long is a day," we encounter two different definitions: one based on the quantum mechanics mechanism of atomic clocks, and the other based on astronomy. These two definitions do not always match precisely.

To solve this problem, *Coordinated Universal Time (UTC)*⁶ was introduced in 1972. It is based on atomic clocks and defines the world time standard, consisting of two parts:

- Atomic time TAI, which is the weighted average of hundreds of atomic clocks worldwide.
- It is based on atomic time and uses micro-adjustments to stay roughly synchronized with *Universal Time (UT)*.

To keep UTC within 0.9 seconds of mean solar time, the *International Earth Rotation and Reference Systems Service (IERS)* inserts or deletes *leap seconds*⁷ as needed. Leap seconds are usually implemented on the last second of June 30 or December 31, adjusting UTC time to

³https://en.wikipedia.org/wiki/Atomic_clock

⁴https://en.wikipedia.org/wiki/International_Atomic_Time

⁵<https://www.space.com/astronomy/earth/mysterious-boost-to-earths-spin-will-make-au>

⁶https://en.wikipedia.org/wiki/Coordinated_Universal_Time

⁷https://en.wikipedia.org/wiki/Leap_second

23:59:60. This adjustment allows UTC to maintain the precision of atomic time while *roughly* synchronizing with the “solar time” of Earth’s rotation. If a leap second adjustment is needed, the relevant institutions announce the adjustment time for the following year one year in advance. For example, in December 2016, the National Time Service Center of China issued the “[Leap Second Announcement](#)”, stating:

On July 6, 2016, the Paris-based International Earth Rotation Service (IERS) issued a leap second announcement in its Bulletin C 52 to global institutions responsible for standard time measurement and dissemination: the international standard time UTC (Coordinated Universal Time) will implement a positive leap second on January 1, 2017, adding 1 second. On July 11, the Time Department of the International Bureau of Weights and Measures (BIPM) issued a leap second adjustment forecast to global timekeeping laboratories participating in the International Atomic Time (TAI) calculation. Due to time zone differences, China will synchronize the leap second adjustment with the world at 07:59:59 Beijing time on January 1, 2017, resulting in the special phenomenon of 07:59:60.

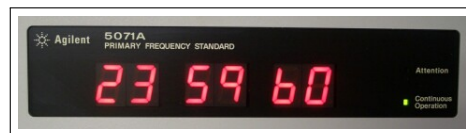


Figure 3.5: Seeing a time like this again should no longer be surprising—it indicates that this second is a leap second. Image from leapsecond.com



The abbreviation for Coordinated Universal Time is UTC, which is clearly not an acronym for “Coordinated Universal Time.” Behind this is a story of compromise: the International Telecommunication Union wanted Coordinated Universal Time to have a single abbreviation in all languages. People in English and French-speaking regions both wanted their respective abbreviations—CUT and TUC—to become the international standard, and the final compromise was to use UTC.

In the military, the Coordinated Universal Time zone is represented by “Z.” In aviation, all times used are uniformly specified as Coordinated Universal Time. Moreover, Z should be read as “Zulu” in the NATO phonetic alphabet over radio, so Coordinated Universal Time is also called “Zulu time.” For example, if a plane takes off at 18:00 Beijing time (UTC+8), it would be written as 1000z, or read as “1000 Zulu.”

UTC unified the world time standard, and time zones in different countries and regions are also defined in terms of UTC. If the local time is ahead of UTC, for example, China's time is 8 hours ahead of UTC, it is written as *UTC+8*, commonly known as the *East Eighth Zone*. Conversely, if the local time is behind UTC, for example, Hawaii's time is 10 hours behind UTC, it is written as *UTC-10*, commonly known as the *West Tenth Zone*.

The appearance of leap seconds introduces complexity and potential problems for some computer programs. According to the concept of leap seconds, a day does not always have exactly 86,400 seconds; it may have 86,399 seconds (deleting a leap second) or 86,401 seconds (adding a leap second). Leap seconds bring new problems to time measurement in computer systems. To understand this, we first need to understand the clock system in computer systems.

3.2.2 Wall Clocks and Monotonic Clocks

Modern computer systems have two clock systems: the *wall clock* and the *monotonic clock*. These two clock systems answer different questions. Understanding the difference between these two clock systems helps deepen the understanding of time issues in distributed systems.

Wall Clock

As the name suggests, the “wall clock” can be understood as a clock hanging on the wall; it answers the question “what time is it now?” The most commonly used *Unix time*⁸ in the computer field represents the number of seconds elapsed since January 1, 1970. Under Linux, you can use `clock_gettime(CLOCK_REALTIME)` to obtain the system's Unix time; similarly, Java provides `System.currentTimeMillis()` for the same purpose.

Unix time does not account for leap seconds, so any code using Unix time to measure time differences must additionally consider whether leap seconds need to be taken into account. Historically, leap seconds have caused many major failures in the Internet domain. For example, on June 30, 2012⁹, an extra second in global time caused chaos on the Internet. Mainstream platforms such as LinkedIn, Reddit, and Mozilla experienced partial outages, and countless Linux servers (including many servers from Red Hat, Debian, and Ubuntu vendors) saw CPU utilization spike to 100%, causing crashes. For more leap-second-related issues and failures, please refer to [11].

⁸https://en.wikipedia.org/wiki/Unix_time

⁹<https://www.linkedin.com/pulse/2012-linux-leap-second-buga-system-design-lesson-ha>



From the definition of Unix time, we know that the unit of this time representation is seconds, but some APIs provide Unix time in other units. For example, Java's `System.currentTimeMillis()` function provides the number of milliseconds elapsed since January 1, 1970.

In addition to the leap second problem, we will also see later that when using the NTP protocol for time synchronization, the wall clock can also jump backward.

Monotonic Clock

Unlike the wall clock, the monotonic clock answers the question of time difference, i.e., “how much time has passed.” It is like a stopwatch in a computer system, telling developers how long the system has been running. It represents the time elapsed since “some arbitrary point in time” (usually the moment the operating system boots). The absolute value itself (e.g., 43,567,890 nanoseconds) has no meaning; only the difference between two monotonic clock readings is meaningful.

Because the monotonic clock satisfies the property of monotonic increase, no matter how the NTP protocol adjusts the system time, the monotonic clock only moves forward and will not go backward. Even if the NTP protocol discovers that the local clock is running too fast, it will not simply roll the monotonic clock back; instead, it will subtly adjust the clock frequency (slewing) to make it run slightly slower until it catches up with the standard time, but the monotonic clock value will still increase.

It is important to emphasize that since the starting time of the monotonic clock is usually the operating system boot time, monotonic clocks on two different machines cannot be compared. That is, the monotonic clock cannot be compared across nodes.

With this understanding of the two clock systems, we can next look at how systems synchronize physical time.

3.2.3 Physical Time Synchronization

Atomic clocks built on quantum physics are more accurate but expensive. To solve the problem of low time precision on computers, computers periodically synchronize time with high-precision time systems through clock synchronization protocols. *NTP (Network Time Protocol)*¹⁰ is currently the most mainstream clock synchronization protocol; almost all personal computers, smart devices, and mobile devices use NTP for clock synchronization.

¹⁰https://en.wikipedia.org/wiki/Network_Time_Protocol

The NTP protocol organizes its structure in multiple layers, where each layer is called a *stratum*. The stratum number of an NTP server that obtains clock synchronization from the most authoritative clock is usually set to stratum 1, serving as the primary time server to provide clock synchronization for other devices on the network. Stratum 2 obtains time from stratum 1; similarly, stratum 3 obtains time from stratum 2, and so on. The clock stratum value ranges from 1 to 16; the smaller the value, the higher the time precision.

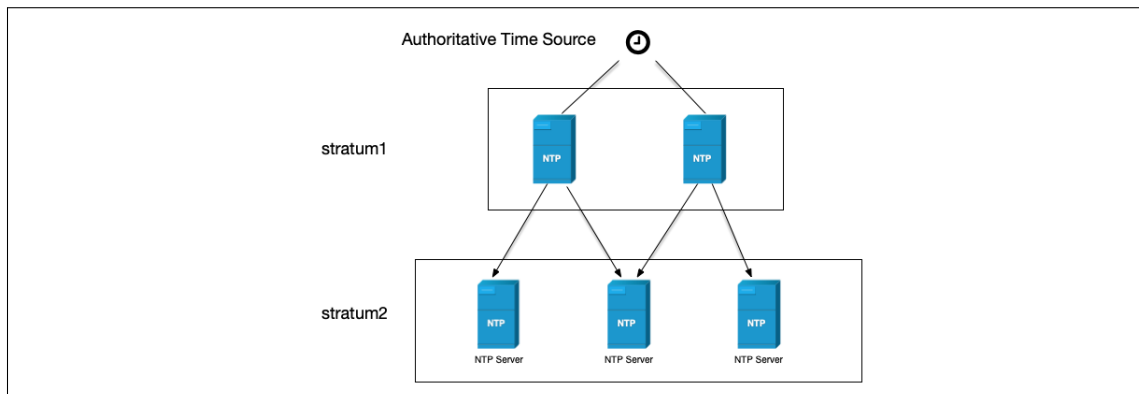


Figure 3.6: The hierarchical model structure of the NTP protocol

The NTP protocol is a typical client/server architecture protocol. After receiving a request from an NTP client, the NTP server responds with the current NTP server time to correct the NTP client time. Due to network delays, differences in CPU processing speed, and other reasons, to precisely synchronize the clock, the response sent to the NTP client needs to include the times when the server received and sent the response, so that the client can estimate the delay.

Figure 3.7 shows a complete NTP request process:

1. The client sends an NTP request packet to the server at time t_1 , which includes the departure time t_1 of this packet from the client.
2. The NTP request packet arrives at the server at time t_2 . After receiving and processing the packet, the server sends a response packet at time t_3 . The response packet carries the departure time t_1 of the packet from the client, the time t_2 when the server received the request packet, and the departure time t_3 of the response packet from the server.
3. The client receives the response packet from the server at time t_4 .

As can be seen, if we can calculate the delay time difference between the client and the server, then based on the server response time t_3 , we can infer the current time at the client.

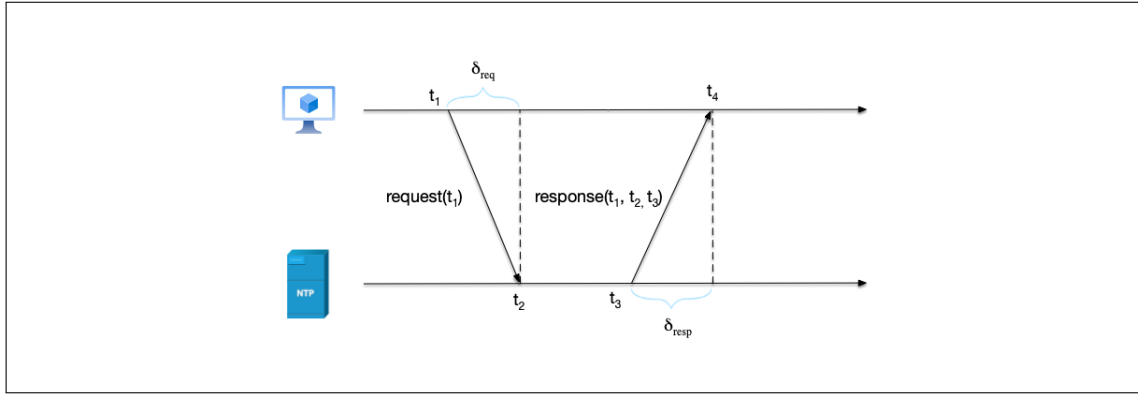


Figure 3.7: The NTP protocol flow

In the process above, there are two time differences related to network delay:

$$\delta_{req} = t_2 - t_1, \text{ request delay}$$

$$\delta_{resp} = t_4 - t_3, \text{ response delay}$$

The total round-trip delay is the sum of the two:

$$\delta = \delta_{req} + \delta_{resp}$$

The NTP protocol assumes that the round-trip time difference divided by 2 is the delay time of one NTP message. Adding this time to the server response time t_3 gives the time at the NTP client. That is, the time θ that the NTP client should set after receiving the response is:

$$\theta = t_3 + \frac{\delta}{2} = t_3 + \frac{\delta_{req} + \delta_{resp}}{2}$$

Looking closely at the NTP time offset formula $\theta = t_3 + \frac{\delta}{2}$, we will find that this formula hides a huge assumption: the delay of the request and the delay of the response are equal, i.e., $\delta_{req} = \delta_{resp}$. The NTP protocol assumes that the network is symmetric: the outbound and return paths are the same and take the same amount of time. However, in actual wide area network (public Internet) environments, this assumption is often not valid. This is network asymmetry:

- Different routing paths: The Internet's complex routing strategies mean that the path of a data packet from A to B is often different from the path from B back to A. The outbound path may go over fiber optics, while the return path may take a detour or even go through congested links.
- Different bandwidth and queuing: Even if the paths are the same, the inconsistency between upstream and downstream bandwidth (such as common residential broadband

ADSL or cloud server bandwidth limits) can cause the request and response to wait in router queues for vastly different amounts of time.

This asymmetry introduces an irremovable systematic error into NTP. If the request takes 100ms and the response only takes 10ms, NTP crudely assumes that each one-way trip is 55ms, which directly causes the calculated time θ to deviate by 45ms. In addition, NTP's stratum hierarchy further amplifies this uncertainty. A server at stratum 3 needs to synchronize from stratum 2, which in turn synchronizes from stratum 1. Synchronization at each level carries the risk of network asymmetry, and this error accumulates level by level. It is like a game of telephone: the more people the message passes through, the more distorted the "time" heard at the end.

It is precisely because of these unavoidable physical characteristics in physical networks that we can never obtain a truly accurate global physical clock in a distributed system.

In addition to errors, we must also see that as NTP clients, computers have their time adjusted by the NTP protocol. For example, at one point in time, the system clock is synchronized with NTP time, but at another point, due to network delays and other reasons, the estimated network delay differs from before, causing the system to suddenly jump forward or backward in time. This phenomenon of sudden forward or backward time adjustments in NTP clients can also cause problems in production. Cloudflare reported an incident in January 2017¹¹. The core cause of the failure was that some key logic relied on the result of time difference calculations without considering that time could be adjusted backward (i.e., without guaranteeing monotonicity).

From the above analysis, we can see that if we are only interested in measuring time differences (e.g., measuring request-response time differences), we can consider using a *monotonic clock*. The monotonic clock does not require global synchronization, always moves forward, and represents the duration since the host booted, so there will be no time rollback. In contrast, the *wall clock* synchronizes time with the NTP server and may jump backward after time calibration.

3.2.4 Deficiencies of Physical Clocks

So far, we have learned about the several different sources of physical time in computer systems, two different physical clock systems, and the physical time synchronization mechanism. Physical time has two deficiencies: *clock skew* and *clock drift*, which make it impossible to use physical time to determine the order of distributed events in a distributed system.

Clock skew refers to the time difference between the clocks of multiple devices. For example,

¹¹<https://blog.cloudflare.com/how-and-why-the-leap-second-affected-cloudflare-dns/>

in a distributed system, node A's clock shows 12:00:00 while node B shows 12:00:01—this is the clock skew between these two nodes.

Due to clock skew, on different physical devices, we cannot determine the order of events by comparing their respective wall clock times. As shown in Figure 3.8, a message sent by user A at local time 10:01:54.765 is received by user B at local time 10:01:54.432. To user B, it seems as if a message from the “future” has been received.

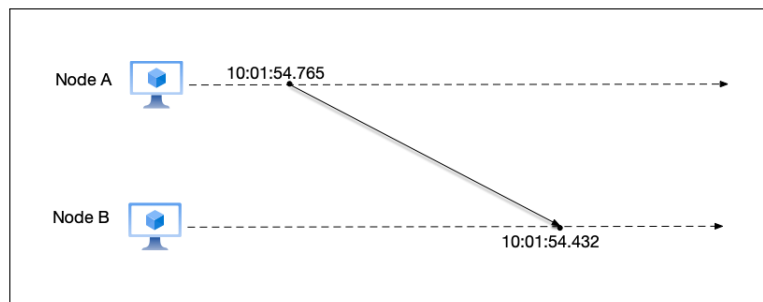


Figure 3.8: Due to the skew between the physical clocks of two nodes, user B appears to have received a message from the “future.”

Clock drift refers to the difference in speed between a device's local clock and standard time. Over time, the local clock's timing speed may be slightly faster or slower, causing the gap with standard time to grow larger and larger. For example, a device's clock may gain or lose a few milliseconds per day; after long-term accumulation, this may lead to minute-level deviations. To correct the device's local clock, it needs to synchronize time with an NTP server. However, during time correction, the time may drift forward or backward.

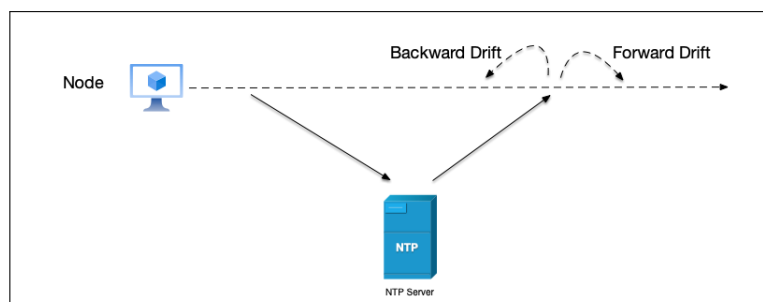


Figure 3.9: After time synchronization and correction with NTP, nodes may drift forward or backward in time.

Since wall clock time may drift backward at random moments, wall clocks cannot be used to measure the time difference between two events in a system; monotonic clocks should be used instead.

At the same time, because different nodes synchronize and correct time with the NTP server, they may have inconsistent wall clock times due to various reasons (network, delay, the node's own clock frequency, etc.). Therefore, in a distributed system composed of multiple nodes, there is no globally unique clock, and we cannot use physical time to measure the order of events as we would in a single-node system.

So, how do we solve the problem of event ordering in distributed systems? We need to re-examine the relationships between events in distributed systems.



It is worth noting that although NTP and ordinary quartz clocks cannot meet the requirements of strongly consistent systems, hardware-assisted clock synchronization technology has made breakthrough progress in recent years. The most famous example is the TrueTime API used by Google Spanner. It uses GPS receivers and atomic clocks in data centers to compress clock errors into an extremely small millisecond-level range, and most critically—it can explicitly tell the caller the upper and lower bounds of this error.

This design of “incorporating the error of physical time into causal inference” allows Spanner to achieve external consistency without completely relying on logical clocks. TrueTime represents another philosophy of distributed system design: hardware and software co-design. We will explore this technology in depth when we discuss distributed transactions later (Section ??). But before that, we need to first master the more universally applicable foundational tool: the logical clock.

3.3 Causality and Event Ordering

Having understood physical clocks, we now return to distributed systems and look at a typical problem. Here, message m1 is sent by user A and broadcast to users B and C, with the content “Where shall we go today?”; message m2 is a reply from user B after receiving message m1, broadcast to users A and C, with the content “Let’s go to the movies.”

From the relationship between m1 and m2, the two are causally related: that is, user A’s question (message m1) comes first, and then user B’s response (message m2). However, user C first sees message m2 and then message m1. This order violates causality: because the response is seen before the question.

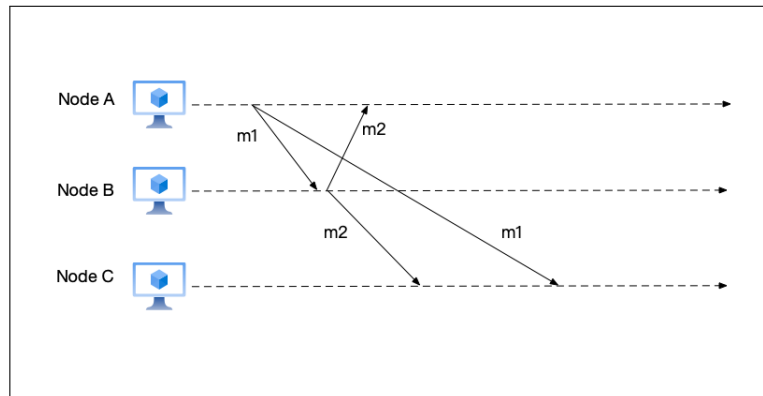


Figure 3.10: Account C sees two messages that violate causality

Let us change this to a more technical example. Suppose message m1 is an instruction to create a new object, and message m2 is an instruction to update this object's value. Clearly, message m1 should occur before message m2, because the object must be created before its value can be modified. If the order of these two messages is reversed on some node, the update operation will fail because the message to create the object has not yet arrived, so the object does not yet exist on that node; and when the message to create the object is received and executed, the object's value will never be updated because the instruction to update the object was already processed earlier.

Returning to the problem, we consider adding a timestamp to each message: the original protocol format was "(message body)", and now it is changed to "(timestamp, message body)". We attempt to solve the causal ordering problem here by adding timestamps to the protocol. So, how should this timestamp be chosen?

First, we rule out the monotonic clock option, because monotonic clock values cannot be compared across different nodes.

Using the wall clock also won't work. Even if a protocol like NTP is used to synchronize time across different nodes, especially when network delays between different nodes and the NTP server are asymmetric, there is no guarantee that the wall clocks of multiple nodes will be consistent. As shown in the figure below, the time when user A sends message m1 is t1, and the time when user B sends message m2 is t2. Here, we cannot guarantee that time t1 on A must be less than time t2 on user B. Therefore, if the wall clock is used to order events, errors may still occur.

In 1975, Paul Johnson and Bob Thomas published a paper [12], also attempting to solve the above problem with timestamps. Leslie Lamport¹² quickly discovered the problem in the paper.

¹²https://en.wikipedia.org/wiki/Leslie_Lamport

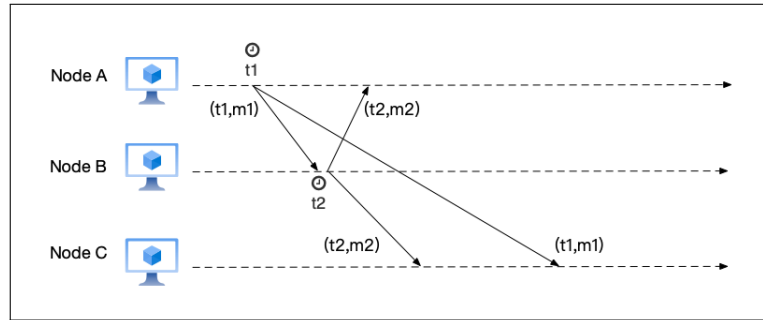


Figure 3.11: Message passing after adding timestamps. Since we cannot guarantee that time t_2 on B is greater than time t_1 on A, errors may still occur.

In Lamport's own words:

I realized that the essence of Johnson and Thomas's algorithm was the use of timestamps to provide a total ordering of events that was consistent with the causal order.

Lamport wrote these insights into the paper "Time, Clocks, and the Ordering of Events in a Distributed System" [13]. Let us see how this paper solves the problem of event ordering.

In the paper, Lamport introduced the **Happened-before** relation¹³, used to express the ordering of multi-node events in distributed systems. Event A happening before event B is denoted as $A \rightarrow B$. Its core idea is to divide events in a distributed system into the following three categories: local events, message send events, and message receive events, and to define precedence among these three categories of events.

The following are the three conditions that define the Happened-before relation:

1. Local events: if events A and B occur sequentially on the same node, then $A \rightarrow B$.
2. Send message event: if event A is a message send event on one node, and event B is the corresponding message receive event on the receiving node, then $A \rightarrow B$. This is because causally related events must have an order: the cause (message send event) comes before the effect (message receive event).
3. Satisfies transitivity: for three events A, B, and C, if both $A \rightarrow B$ and $B \rightarrow C$ hold, then $A \rightarrow C$ also holds.

If for two events A and B, neither $A \rightarrow B$ nor $B \rightarrow A$ holds, then these two events are called **concurrent events**, denoted as $A || B$. In mathematical language, concurrent events are

¹³<https://en.wikipedia.org/wiki/Happened-before>

expressed as:

$$a||b \iff \neg(a \rightarrow b) \wedge \neg(b \rightarrow a)$$

From the three conditions of Happened-before above, it can be seen that two concurrent events must be located on different nodes, and there is no message passing between them. Because there is no message passing, their order relationship cannot be confirmed.

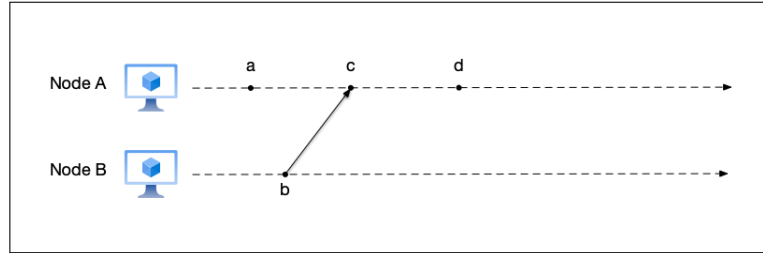


Figure 3.12: An example illustrating the ordering of events among multiple nodes. In the figure, events a and b are concurrent events; it cannot be determined which event occurred first.

Taking Figure 3.12 as an example to explain event ordering, the line and arrow between events b and c in the figure indicate that they are the send and receive events of the same message:

- According to rule 1, events a, c, and d occur sequentially on node A, so $a \rightarrow c$, $c \rightarrow d$;
- According to rule 2, event b is the message send event on node B sending a message to node A, and event c is the corresponding message receive event on node A, so $b \rightarrow c$;
- According to rule 3, $a \rightarrow d$, $b \rightarrow d$;
- For events a and b, neither $a \rightarrow b$ nor $b \rightarrow a$ holds, so they are concurrent events.

Let us expand the figure above by introducing several nodes called “observers.” These observer nodes can see events on nodes A and B, and sort these events on the same observer node’s timeline, which is equivalent to “merging” the event timelines of multiple nodes onto the observer node’s timeline.

In Figure 3.13:

- Node C observes the event sequence (a,b,c,d); this ordering does not violate the Happened-before relation.
- Node D observes the event sequence (b,a,c,d); this ordering does not violate the Happened-before relation.
- Node E observes the event sequence (b,a,d,c); here event d is placed before event c. We have already mentioned earlier that these two events satisfy $c \rightarrow d$, so this ordering is incorrect.

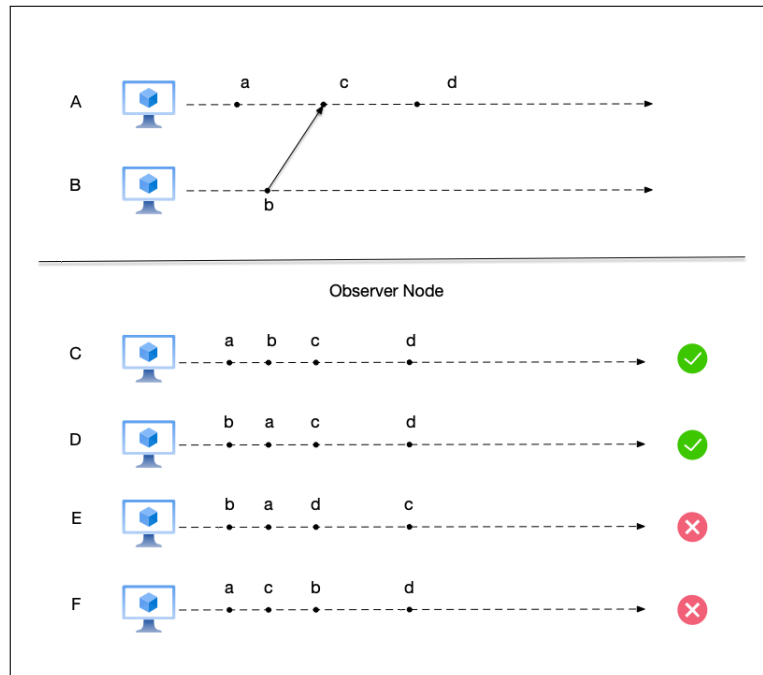


Figure 3.13: Event ordering seen after introducing observers.

- Node F observes the event sequence (a,c,b,d); here event c is placed before event b. We have already mentioned earlier that these two events satisfy $b \rightarrow c$, so this ordering is incorrect.

It is worth explaining here: the difference between node C and node D is how events a and b are ordered. As mentioned earlier, these two events are concurrent events. In some consistency models, different nodes may arrange concurrent events in any order without causing data corruption; we will return to this discussion when we talk about consistency models later.



Social media products can be used to understand the “merging” operation above. Observer nodes and nodes that produce events are equivalent to accounts on social media. Each observer account follows multiple accounts (nodes A and B in the above example). These accounts post different messages on their own timelines, but from the perspective of their followers, these messages ultimately need to be aggregated and have a *reasonable* ordering on the follower’s own timeline.

For concurrent events, the order seen on the same account can be arranged arbitrarily, but for events satisfying the Happened-before relation, their order must be guaranteed.

3.4 Total Order and Partial Order

In the previous section, we introduced the Happened-before relation for events. There also exist concurrent events that do not satisfy this relation. To understand these relationships more deeply, we need to understand two mathematical concepts: *partial order*¹⁴ and *total order*¹⁵.

Many engineers feel intimidated when they see mathematical definitions, but in fact, these two concepts are everywhere in our daily lives. Before entering strict mathematical descriptions, let us first use two life scenarios to build intuitive understanding.

Total Order: The Line at a Bank Counter

Imagine you are standing in line at a bank counter to conduct business. People stand in a sequence. For any two people in the line (say, Alice and Bob), we can definitely determine their order:

- Either Alice is in front of Bob;
- Or Bob is in front of Alice.

This property, where any two elements can be compared, defines a *total order*. In a single-node system, because there is only one CPU clock, all events are like people in a line: their absolute order can always be determined. Therefore, single-node events usually satisfy a total order relationship.

Partial Order: The Reporting Structure of a Company

Now, let us switch the scenario to the organizational chart of a large company. We define a relationship called “reports to” (i.e., a subordinate reports to a superior).

- Vertical comparison (comparable): Programmer Tim reports to the tech manager, and the tech manager reports to the CTO. In this chain, we can clearly say: “Tim is ‘below’ the CTO in the hierarchy.” This relationship is clear.
- Horizontal comparison (incomparable): Now let us use this “reports to” relationship to compare the tech manager and the finance manager. You will find that they cannot be compared—the tech manager does not report to the finance manager, and the finance manager does not report to the tech manager. They work in parallel in their respective departments, with no subordination between them.

In this system, only “some” members have a clear hierarchical relationship, while other members (such as managers of the same level in different departments) cannot be compared through this relationship. This relationship of “order exists only among some members, and

¹⁴https://en.wikipedia.org/wiki/Partially_ordered_set

¹⁵https://en.wikipedia.org/wiki/Total_order

members that cannot be compared concurrently are allowed” is the **partial order**.

Mapping to distributed systems:

- Reporting relationship $\iff$ Happened-before.
- Cross-department managers with no subordination $\iff$ concurrent events.

Understanding this, let us now see how mathematicians describe these two types of relationships in rigorous language.

Partial Order

A partial order represents a binary relation R in which only *some* members of the set can be compared using this binary relation. The binary relation R used to compare set members must satisfy the following three rules in mathematics:

1. Reflexivity: for any member x in the set, xRx holds;
2. Antisymmetry: for any members x and y in the set, if xRy and yRx hold, then $x = y$;
3. Transitivity: for any members x, y , and z in the set, if xRy and yRz hold, then xRz holds.

Let us give an example to understand partial order. For the set $\{a, b, c\}$, the set of all its subsets is

$$\{\{\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}.$$

All these members are subsets of the original set and can be related through the $\subseteq$ relation:

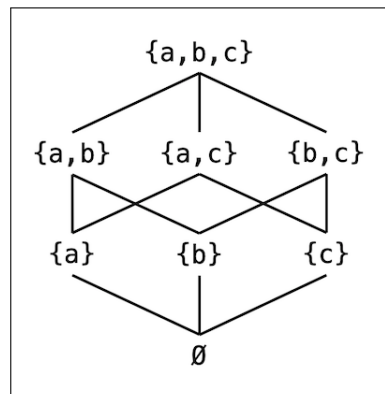


Figure 3.14: The $\subseteq$ relation is a partial order on the subsets of $\{a, b, c\}$

We note that the $\subseteq$ relation here satisfies the three rules above. However, not all members here satisfy the $\subseteq$ relation with each other; for example, $\{a, b\} \not\subseteq \{b, c\}$ and $\{b, c\} \not\subseteq \{a, b\}$.

Tree relationships are also a common form of partial order. In the tree shown in Figure 3.15, nodes at the same level cannot be compared in size, because in terms of the “level” relationship, nodes at the same level are the same; for example, node A and node B have the same level.

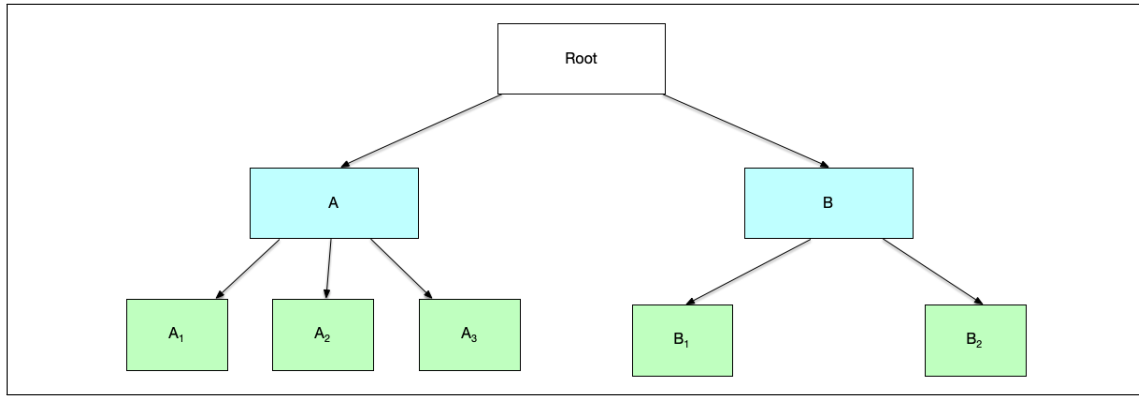


Figure 3.15: Tree relationships are also a form of partial order. Nodes at the same level have the same level and cannot be compared in size.

Total Order

On the other hand, if *all* members of a set can be compared using a binary relation, this binary relation is called a **total order relation**. Formally, a total order relation adds a *totality constraint* on top of a partial order relation. That is, a total order relation R requires satisfying the following four rules:

1. Reflexivity: for any member x in the set, xRx holds;
2. Antisymmetry: for any members x and y in the set, if xRy and yRx hold, then $x = y$;
3. Transitivity: for any members x , y , and z in the set, if xRy and yRz hold, then xRz holds.
4. Totality: for any members x and y in the set, either $x \leq y$ or $y \leq x$ holds.

For example, any two elements in the set of integers can be compared using the binary relation $\leq$, and this relation is a total order relation for this set.

We can see that the difference between partial order and total order is: partial order requires elements in the set to be *partially* ordered, while total order requires elements in the set to be *totally* ordered.

From the above explanations of total order and partial order, we can see that the Happened-before relation is a partial order relation. Those concurrent events are events whose order cannot be compared.



However, the Happened-before relation does not satisfy reflexivity, because for an event A, one cannot say “A happened before A.” So in the strict sense, the Happened-before relation is called an *irreflexive partial order*. For simplicity, we will still consider the Happened-before relation as a partial order relation later and will no longer specifically emphasize the violation of reflexivity.

On a single-node system, because there is a unique global clock, events on the system occur sequentially in time order; one event ends before the next event occurs. Therefore, it is easy to perform a total ordering of events on a single-node system. However, on a distributed system, performing a total ordering of events is very difficult. There are two reasons:

1. Events on multiple nodes occur concurrently;
2. There is no global clock in a distributed system; instead, each node has its own clock.

Taking Figure 3.16 as an example, we represent events using start and end times as “[start time, end time].” We can see that in a single-node system, events execute sequentially and satisfy a total order relationship; in a distributed system, events on multiple nodes execute concurrently. For example, event [1,3] on node A and event [2,4] on node B do not satisfy a total order relationship.

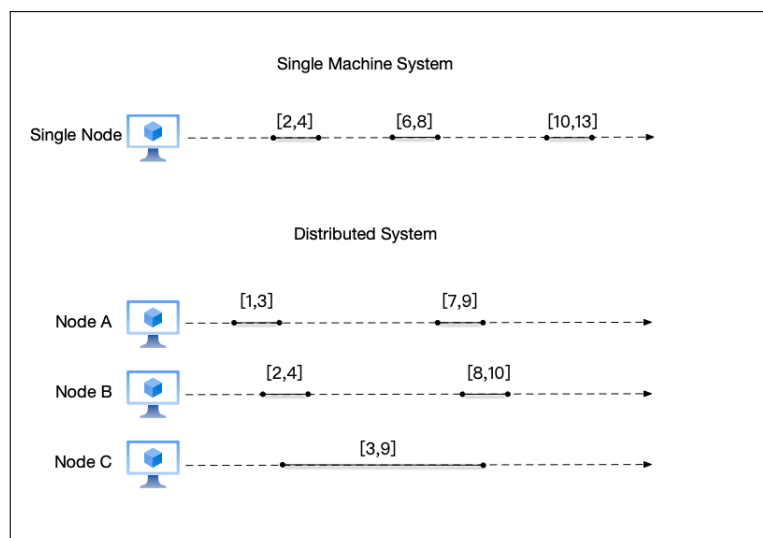


Figure 3.16: single-node systems and distributed systems

The figure above also implies an assumption: there exists a *global clock* for the distributed system, and all nodes in the system can record the start and end times of events based on this

global clock. In reality, maintaining this global physical clock for each node is generally handled by services like NTP. However, considering the network delay between each node and the NTP service, as well as the operating conditions of the nodes, the time synchronization between this global clock and each node has different errors. Therefore, on a distributed system, there is no global clock; instead, each node has its own clock. This is the second reason why global ordering is difficult on a distributed system.

In daily life, we are already accustomed to the events around us being totally ordered. for example,

- Waiting in line to buy movie tickets, or waiting to pass through a traffic light;
- When shopping, first putting items into the shopping cart, then checking out.

However, when it comes to multi-person interactions, there are many scenarios where it is difficult to totally order events, such as:

- In a group chat, multiple users are speaking. The order of these messages may be different in different accounts;
- Multiple comments on the same post on social media may also not have a single ordering.

The reason people are accustomed to the events around them being ordered is mainly due to the single perspective of an individual. But from a distributed perspective, not all events occurring on different nodes are ordered.

We will see later in the consistency model chapter that in many scenarios there is no need to implement a total order relationship. Maintaining a total order relationship among events comes at a great cost: more resource consumption, less event concurrency, and so on. On the contrary, in many cases, implementing *causal consistency* [Section ??] is sufficient—that is, only maintaining the order for causally related events, while concurrent events among multiple nodes can be ordered arbitrarily. We will continue to explore this problem in depth in the consistency models [Section ??].

3.5 Causality and the Happened-before Relation

Causality in real life is determined based on loosely synchronized clocks (watches, wall clocks) under the illusion that a global clock exists. for example,

- Making time plans: plan to arrive at the station at 6:10 to catch the 6:30 bus;
- Alibi¹⁶: if a suspect appears in a video at some location at 6:00, then they cannot have committed a crime ten kilometers away at 6:10.

¹⁶<https://en.wikipedia.org/wiki/Alibi>

In these life scenarios, the requirements for time precision are relatively coarse, tolerating errors of several seconds or even tens of seconds. This is because in real life, information flows slowly, so small clock errors do not cause serious problems. But in a distributed system, events occur at higher frequencies and speeds, and the duration of events is many orders of magnitude shorter than events in real life. If the physical clocks in a distributed system are not precisely synchronized, it is difficult to determine the causal relationships between events.

Let us review the three situations that satisfy the Happened-before relation. The second situation implicitly contains causality: the message send event is the cause, and the message receive event is the effect. Happened-before is a way to reason about causality in distributed systems. In the second situation of Happened-before, it considers whether information *flows* from one event to another, which indicates whether one event may affect another. Therefore, the Happened-before relation can conveniently express causality between events: if event A *causes* event B, then event A *must* have happened before event B.

But the other two situations of Happened-before do not necessarily imply causality. Situation one says that events occurring sequentially within the same node have a Happened-before relationship; situation three says that the Happened-before relation is transitive. In these two situations, events satisfying the Happened-before relationship do not necessarily have a causal relationship. Therefore, we say that if event A happened before event B, then event A *might* have caused event B.

Finally, for concurrent events, there *must* be no causal relationship. For example, events a and b in Figure 3.12.

To summarize, causality and the Happened-before relation have the following three relationships:

- If event A causes event B, then event A *must* have happened before event B (if event A causes event B, then $A \rightarrow B$);
- If event A happened before event B, then event A *might* have caused event B (if $A \rightarrow B$, then A might cause B);
- For two concurrent events, there is definitively no causal relationship (if $A || B$, then event A MUST NOT cause event B, and event B MUST NOT cause event A).



Please note in the above statements which ones use *might* and which ones use *must*.

3.6 Logical Clocks

Since physical time cannot be used as a basis for measuring the order of events in distributed systems, Lamport then introduced the concept of “*logical clocks*” in his paper to solve this problem.

A logical clock can be understood as a counter, with each event corresponding to a separate count. There are many ways to implement this count, but no matter which one, it must satisfy the following *clock condition* (let the logical clock function for events be $C(e)$, where parameter e is the event):

If event a happens before event b , then $C(a) < C(b)$.

When explaining Happened-before earlier, we used the symbol $a \rightarrow b$ to denote that event a happens before event b . Therefore, the clock condition can be expressed in mathematical language as follows:

$$a \rightarrow b \Rightarrow C(a) < C(b)$$

Note that in logical clocks, the converse of the clock condition does not hold, i.e.:

If $C(a) < C(b)$, it is not certain that event a happens before event b .

In mathematical language:

$$C(a) < C(b) \not\Rightarrow a \rightarrow b$$

Note that “it is not guaranteed that event a happened before event b ” means: it is possible that event a happens before event b , or the two events may be concurrent events. That is: $C(a) < C(b) \Rightarrow (a \rightarrow b) \vee (a \parallel b)$.

If both conditions above can be satisfied simultaneously, i.e., $a \rightarrow b \iff C(a) < C(b)$, this is called the “*strong consistency condition*”.



The “consistency” in the strong consistency condition is different from the “consistency” in the “consistency models” we will mention later. The strong consistency condition means that there is a consistent relationship between logical time and event order: by comparing logical times, the order relationship between events can be derived; conversely, if the order relationship between events is known, the magnitude relationship of their logical times can also be determined.

Although there are many different logical clock implementations, in general, the implementation of logical clocks has two main parts [14]:

- The data structure used to represent the logical clock;
- The protocol used to update the logical clock representation when there is message passing between nodes.

Each node maintains the following data:

- The logical clock within the node, used to measure the order of events within this node;
- The global logical clock from the node's perspective.

The protocol includes the following two rules:

- Rule 1: describes how to update the local logical clock when a node executes an event;
- Rule 2: describes how to update the global logical clock from this node's perspective.

Different logical clock implementations use different data structures to represent logical clocks and have different logic for updating the local and global logical clocks, but in general, they all contain the core points above.

After understanding the concept of logical clocks, let us look at two logical clock implementations: Lamport clocks and vector clocks. Among them, Lamport clocks only satisfy the basic clock condition, while vector clocks satisfy the strong consistency condition.

3.6.1 Lamport Clocks

In Lamport clocks, each node maintains a counter starting from 0 and updates the counter value according to the following rules:

- Rule 1: before executing a local event (events include receiving, sending messages, or local events), increment the counter by 1: $C_i = C_i + 1$. The logical clock corresponding to this event is the updated count;
- Rule 2: when sending a message, perform operations in the following order:
 - According to rule 1, increment the counter, i.e., $C_i = C_i + 1$;
 - Attach the current logical clock of this node to the message body;
- Rule 3: when receiving a message, perform operations in the following order:
 - Use the logical clock in the message to update the local logical clock. The rule is: $C_i = \max(C_i, C_{msg})$;
 - According to rule 1, increment the counter, i.e., $C_i = C_i + 1$.

Let us illustrate the calculation process of Lamport clocks through an example. In the figure below, “LC” denotes Lamport Clock (LC is short for Lamport Clock), uppercase letters denote node names, and an uppercase letter plus a number denotes the *n*th event on that node. For example, $LC(A) = 1$ means that node A’s Lamport clock is currently 1, and $LC(A1) = 2$ means that the Lamport clock of the first event on node A is 2.

1. Initial state As shown in Figure 3.17, at this point the Lamport clock values of nodes A, B, and C are all 0, i.e., $LC(A) = LC(B) = LC(C) = 0$;

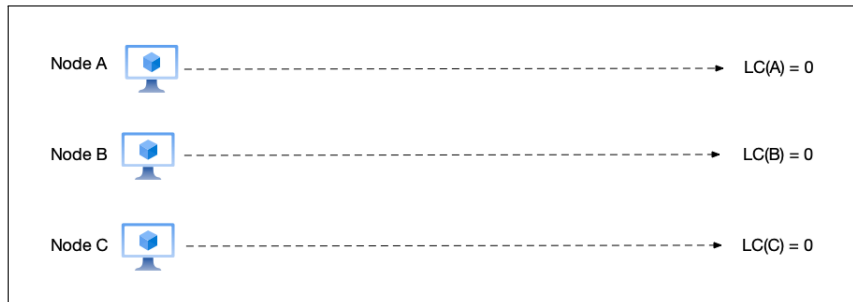


Figure 3.17: Logical clock example 1: initial state

2. Node A sends a message to node B As shown in Figure 3.18,
 - Node A: before executing the send message event, node A increments its counter to $LC(A) = LC(A) + 1 = 1$, then sends the current count value as part of the message to node B. Therefore, the logical clock corresponding to this send message event $LC(A1)$ is 1;
 - Node B: after receiving the message from node A, it updates its local counter to $LC(B) = \max(LC(B), C_{msg}) = 1$, then increments the counter to $LC(B) = LC(B) + 1 = 2$. Therefore, the logical clock corresponding to this receive message event $LC(B1)$ is 2;
 - Node C: a local event occurs on node C, $LC(C1) = 1$.
3. Node B sends a message to node C As shown in Figure 3.19,
 - Node A: because no new event occurs, node A keeps $LC(A) = 1$;
 - Node B: before executing the send message event, node B increments its counter to $LC(B) = LC(B) + 1 = 3$, then sends the current count value as part of the message to node C. Therefore, the logical clock corresponding to this send message event $LC(B2)$ is 3;

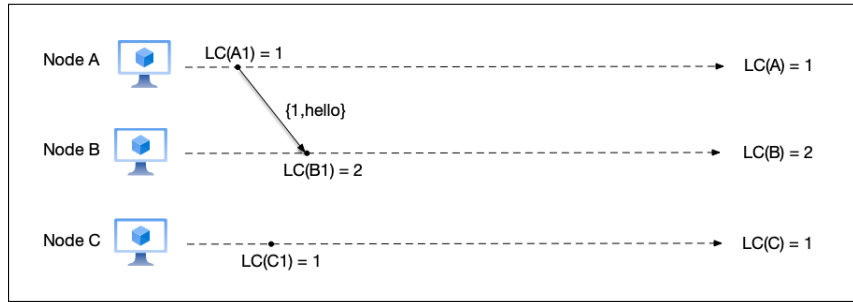


Figure 3.18: Logical clock example 2: node A sends a message to node B

- Node C: after receiving the message from node B, it updates its local counter to $LC(C) = \max(LC(C), C_{msg}) = 3$, then increments the counter to $LC(C) = LC(C) + 1 = 4$. Therefore, the logical clock corresponding to this receive message event LC(C2) is 4.

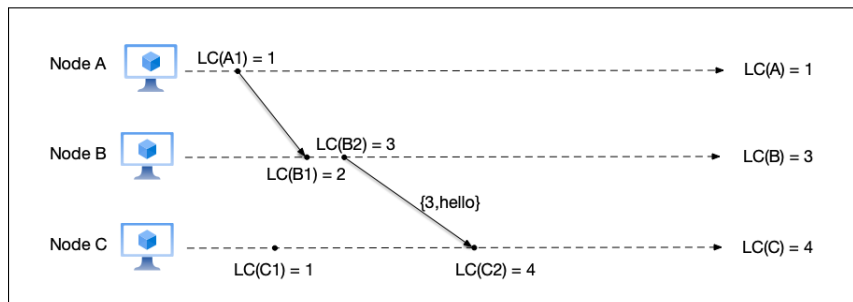


Figure 3.19: Logical clock example 3: node B sends a message to node C

4. Node C sends a message to node A As shown in Figure 3.20,

- Node A: after receiving the message from node C, it updates its local counter to $LC(A) = \max(LC(A), C_{msg}) = 5$, then increments the counter to $LC(A) = LC(A) + 1 = 6$. Therefore, the logical clock corresponding to this receive message event LC(A2) is 6;
- Node B: because no new event occurs, node B keeps $LC(B) = 3$;
- Node C: before executing the send message event, node C increments its counter to $LC(C) = LC(C) + 1 = 5$, then sends the current count value as part of the message to node A. Therefore, the logical clock corresponding to this send message event LC(C3) is 5.

It is easy to see that the calculation rules for Lamport clocks satisfy the clock condition that any logical clock implementation must satisfy. It guarantees the clock condition in the

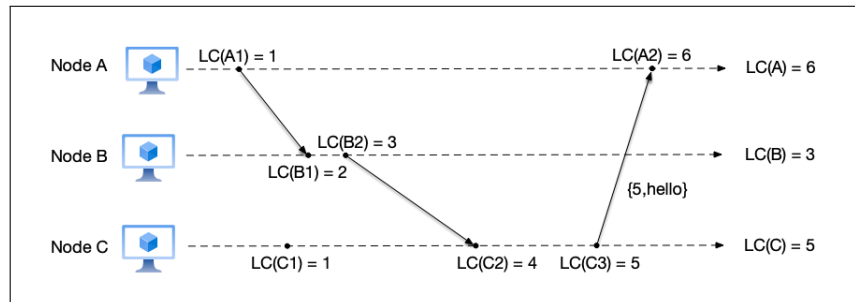


Figure 3.20: Logical clock example 4: node C sends a message to node A

following ways:

- Local events: each time a node executes an event, it increments its local Lamport clock by 1. In this way, events occurring sequentially within the node can reflect their order through the Lamport clock. For example, events B1 and B2 in Figure 3.19, $LC(B1) = 2 < LC(B2) = 3$;
- Message send/receive events: when a node sends a message, it carries the Lamport clock corresponding to that send message in the message body. When receiving a message, it takes the larger of the current local Lamport clock and the message's Lamport clock, then increments by 1 as the receive event time. For example, events B2 and C2 in Figure 3.19 are the corresponding message send and receive events, $LC(B2) = 3 < LC(C2) = 4$.

Conversely, the magnitude of two events' Lamport clocks cannot reflect their order. For example, in the figure, $LC(C1) = 1 < LC(B2) = 3$, but these two events are concurrent events.

It can be seen that although Lamport clocks maintain the clock condition required by logical clocks, they cannot determine the causal relationships between events through this time. The vector clock introduced below can solve this problem.



Although the Lamport clock mentioned in this paper does not satisfy the strong consistency condition, this does not prevent the paper from becoming one of the most important papers in the distributed systems field. In addition to proposing the definition of the Happened-before relation and the calculation method of Lamport logical clocks, the paper more importantly proposes that any distributed system can be described as a state machine composed of events in a specific order. If the multiple nodes that make up a distributed system execute the same events in the same order, they can form the same state machine, and what determines the event order is the logical clock mentioned

here. This idea is also the foundation of consensus algorithms such as Paxos and Raft mentioned later.

3.6.2 Vector Clocks

The problem with Lamport clocks is that using a single integer to simultaneously represent both local logical time and global logical time is insufficient for tracking the causal relationships of events. In Figure 3.20, the reason why $LC(A1)$ and $LC(C1)$ are both 1 is that from the perspectives of nodes A and C, the local logical time and global logical time they see are both 1.

Vector clocks were proposed by Friedemann Mattern in 1988 [15]. Its idea is: each node has its own local logical time, and the logical times of all nodes together form a vector. This time vector is the global logical time. To synchronize this global logical time, when nodes communicate, in addition to synchronizing their own node's time value, they also synchronize the time values of other nodes saved on this node. In this way, a globally unique global logical time can be maintained.

Vector clocks can conveniently be used to track the *causal history* of events. Still taking Figure 3.20 as an example, the causal history of event C2 is: $\{A1, B1, B2, C1\}$. All events in this set happened before event C2. In addition, vector clocks also satisfy the strong consistency condition: if $VC_i[j] < VC_m[n]$, then there must be $e_{i,j} \rightarrow e_{m,n}$; and if two events are concurrent events, it means that the vector clocks of the two events cannot be compared, and vice versa.

Let us now introduce the specific implementation of vector clocks. Each node i maintains a clock vector VC_i , whose length is the number of nodes in the distributed system, and the initial value of the vector is $[0, \dots, 0]$. $VC_i[j]$ represents the logical time of node j saved on node i , where i and j represent node indices. The calculation rules for vector clocks are as follows:

- Rule 1: before executing an event on node i (events include receiving, sending messages, or local events), increment the logical time of this node by 1, i.e., $VC_i[i] = VC_i[i] + 1$;
- Rule 2: when sending a message, perform operations in the following order:
 - According to rule 1: increment the logical time of this node in the vector clock, i.e., $VC_i[i] = VC_i[i] + 1$;
 - Attach this node's vector clock VC_i to the message body;
- Rule 3: when receiving a message, perform operations in the following order:

- Update the local vector clock according to the vector clock in the message. For each component k in the vector, $VC_i[k] = \max(VC_i[k], VC_j[k])$;
- According to rule 1: increment the logical time of this node in the vector clock, i.e., $VC_i[i] = VC_i[i] + 1$;

Because logical clocks are expressed as vectors, we need to define the rules for vector comparison. $VC_i < VC_j$ if and only if the following conditions are satisfied simultaneously:

- For every component k in the vector, $VC_i[k] \leq VC_j[k]$;
- There exists at least one component k such that $VC_i[k] < VC_j[k]$.

For example, $VC_i = [0, 1, 2] < VC_j = [1, 1, 2]$, while vectors $[1, 2, 3]$ and $[1, 3, 2]$ cannot be compared.

As a comparison, let us re-implement the previous Lamport clock example using vector clocks. In the figure below, “VC” denotes vector clock (VC is short for Vector Clock), uppercase letters denote node names, an uppercase letter plus a number denotes the n th event on that node, and “[k ” denotes taking the k th component value of the vector clock. For example, $VC(A) = [1, 0, 0]$ means that node A’s current vector clock is $[1, 0, 0]$, and $VC(A1) = [2, 0, 0]$ means that the vector clock of the first event on node A is $[2, 0, 0]$.

1. Initial state As shown in Figure 3.21, at this point the vector clock values of nodes A, B, and C are all 0, i.e., $VC(A) = VC(B) = VC(C) = [0, 0, 0]$;

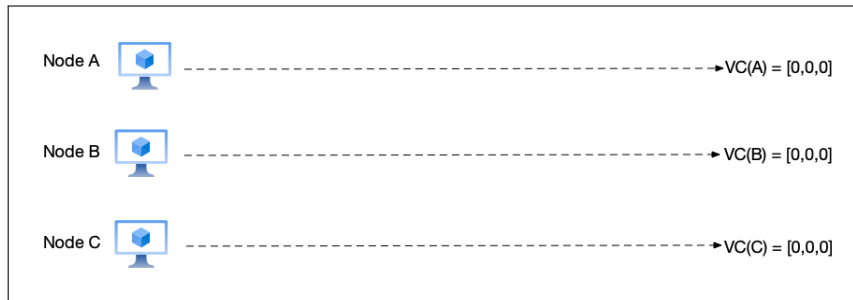


Figure 3.21: Vector clock example 1: initial state

2. Node A sends a message to node B As shown in Figure 3.22,
 - Node A: before executing the send message event, node A increments the vector counter corresponding to this node in the local vector clock, i.e., $VC(A)[1] = VC(A)[1] + 1 = 1$, obtaining the new vector clock $VC(A) = [1, 0, 0]$, then sends

the current vector clock as part of the message to node B. Therefore, the vector clock corresponding to this send message event $VC(A1)$ is $[1,0,0]$;

- Node B: after receiving the message from node A, it updates its local vector counter to $VC(B)[1] = \max(VC(B)[1], VC_{msg}[1]) = 1$, then increments the local vector clock's counter for this node to $VC(B)[2] = VC(B)[2] + 1 = 1$. Therefore, the vector clock corresponding to this receive message event $VC(B1)$ is $[1,1,0]$;
- Node C: a local event occurs on node C, $VC(C1) = [0,0,1]$.

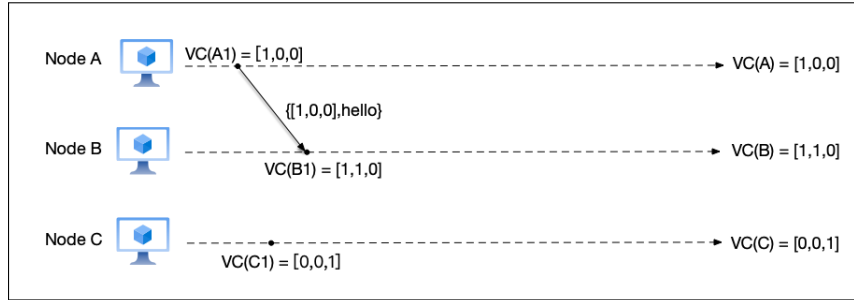


Figure 3.22: Vector clock example 2: node A sends a message to node B

3. Node B sends a message to node C As shown in Figure 3.23,

- Node A: because no new event occurs, node A keeps $VC(A) = [1,0,0]$;
- Node B: before executing the send message event, node B increments its counter to $VC(B)[2] = VC(B)[2] + 1 = 2$, then sends the current vector clock as part of the message to node C. Therefore, the vector clock corresponding to this send message event $VC(B2)$ is $[1,2,0]$;
- Node C: after receiving the message from node B, it updates its local vector counter to $VC(C)[1] = \max(VC(C)[1], VC_{msg}[1]) = 1$ and $VC(C)[2] = \max(VC(C)[2], VC_{msg}[2]) = 2$, then increments its counter to $VC(C)[3] = VC(C)[3] + 1 = 2$. Therefore, the vector clock corresponding to this receive message event $VC(C2)$ is $[1,2,2]$.

4. Node C sends a message to node A As shown in Figure 3.24,

- Node A: after receiving the message from node C, it updates its local vector counter: for each component k in the vector, $VC(A)[k] = \max(VC(A)[k], VC_{msg}[k])$, thus obtaining $VC(A) = [1,2,3]$, then increments the counter to $VC(A)[1] = VC(A)[1] + 1 = 2$. Therefore, the vector clock corresponding to this receive message event $VC(A2)$ is $[2,2,3]$;
- Node B: because no new event occurs, node B keeps $VC(B) = [1,2,0]$;

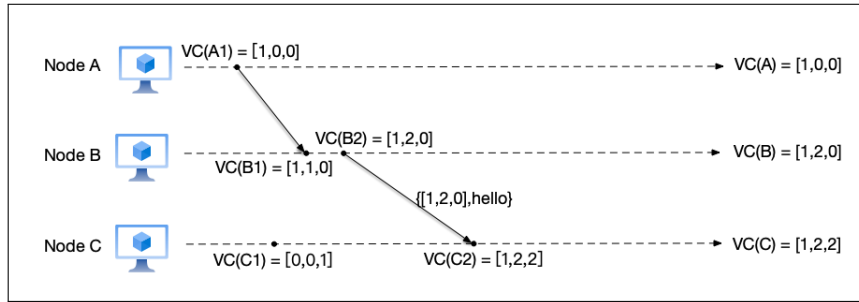


Figure 3.23: Vector clock example 3: node B sends a message to node C

- Node C: before executing the send message event, node C increments its counter to $VC(C)[3] = VC(C)[3] + 1 = 3$, then sends the current count value as part of the message to node A. Therefore, the vector clock corresponding to this send message event $VC(C3)$ is $[1,2,3]$.

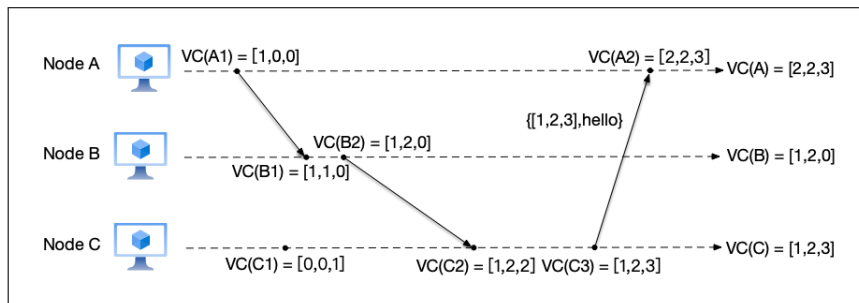


Figure 3.24: Vector clock example 4: node C sends a message to node A

We mentioned earlier that the causal history of event C2 is: $\{A1, B1, B2, C1\}$. All events in the set happened before event C2, because the vector clocks of these events are all smaller than the vector clock of event C2.

In addition, in Figure 3.20, $LC(C1) = 1 < LC(B2) = 3$, but we cannot assert the causal relationship between the two from this. That is, when $LC(a) < LC(b)$, it is possible that event a happened before event b, or the two events may be concurrent events. In the Lamport clock algorithm: $LC(a) < LC(b) \Rightarrow (a \rightarrow b) \vee (a || b)$.

In Figure 3.24, $VC(C1) = [0,0,1]$ and $VC(B2) = [1,2,0]$. According to the vector clock comparison algorithm, the two cannot be compared in magnitude. Therefore, we can assert that the two events are concurrent events.

Although vector clocks perfectly solve the problem that Lamport clocks cannot identify concurrent events and satisfy the strong consistency condition, there is no such thing as a free

lunch. The “perfection” of vector clocks comes at the cost of storage. We notice that the length of the vector clock VC depends on the number of nodes N in the system. This means:

- Network bandwidth consumption: every message transmission needs to carry this vector of length N . If this is in a cluster with hundreds or thousands of nodes, or in a microservices architecture with thousands of service instances, the data volume of this vector may become very large, even exceeding the size of the message body itself, consuming precious network bandwidth.
- Storage space consumption: each node needs to locally maintain and store this ever-growing vector.
- Poor support for dynamics: vector clocks usually assume that the number of nodes in the system is fixed. If the system frequently performs dynamic scaling (nodes going online or offline), maintaining a vector of fixed length will become extremely difficult and inefficient.



We will see in Section ?? how vector clocks are used to guarantee causal ordering between events; and in Section ??, we will introduce in detail how Amazon’s Dynamo system uses vector clocks to resolve data conflicts.

3.7 Global Snapshots in Distributed Systems

As an application of logical clocks, let us see how to obtain global snapshots in distributed systems.

In distributed systems, snapshots have the following uses:

- Fault recovery and checkpointing: computational tasks in distributed systems usually involve multiple nodes, and depend on non-deterministic inputs (such as network requests). If a system failure occurs, snapshots can save the global state of the system, so that after fault recovery, computation can restart from the nearest checkpoint, avoiding recalculation from scratch and reducing waste of computing resources and data loss.
- Consistency checking and global state capture: the state of a distributed system is scattered across multiple nodes. Snapshots can help capture a globally consistent state of the system. This is very useful for debugging, verifying system behavior, or detecting data consistency. For example, through snapshots, we can check whether distributed transactions meet consistency requirements.

- Deadlock detection: in distributed systems, deadlocks may involve mutual waiting among multiple nodes. Snapshots can capture the state of all nodes in the system, helping to detect whether deadlock situations exist and providing a basis for solving the problem.
- Distributed debugging and performance analysis: snapshots can record the state of a distributed system at a certain moment, helping developers analyze the system's operating conditions, locate performance bottlenecks, or logical errors.
- Log compaction and state recovery: in distributed systems, logs may be very large. By periodically creating snapshots, logs can be compacted to the snapshot point, reducing the overhead of log storage, and allowing the system state to be quickly rebuilt based on the snapshot and subsequent logs during recovery.
- Distributed algorithm verification: the correctness of certain distributed algorithms (such as consensus algorithms, distributed locks, etc.) depends on the global state of the system. Snapshots can be used to verify whether these algorithms satisfy expected properties during execution.
- Fault tolerance and high availability: snapshots are one of the key technologies for achieving fault tolerance and high availability. By periodically saving system state, services can be quickly recovered in the event of node failures or network partitions, reducing downtime.

At the beginning of this chapter, we briefly explained the concepts of state, events, and snapshots. With the definition of the Happened-before relation, we can now give the properties that snapshots need to satisfy:

If $A \rightarrow B$, and B is an event saved in the generated snapshot, then event A must also be in snapshot B.

This property requires that snapshots must not have the situation of *missing causal history*: if an event appears in a snapshot, all events that happened before that event should also appear in the snapshot.

Only snapshots that satisfy this property can be called *consistent snapshots*. For example, the snapshot in Figure 3.25 (the snapshot is composed of events within the rectangle) is not a consistent snapshot:

- Case 1: event $A1 \rightarrow B1$, and node B's snapshot has event B1, but event A1 is not in node A's snapshot;
- Case 2: event $B1 \rightarrow B2$, and node B's snapshot has event B2, but event B1 is not in node B's snapshot;

Both cases above belong to the missing causal history problem: some event is recorded in the snapshot, but some other events that happened before it are not recorded in the snapshot.

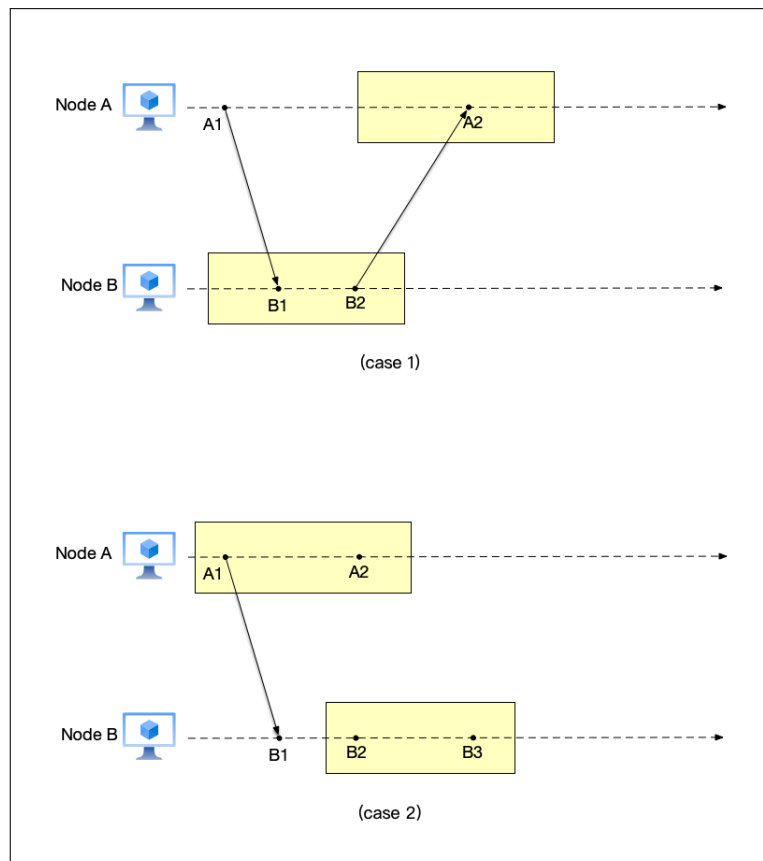


Figure 3.25: Example of an inconsistent global snapshot with missing causal history

If a system runs as a single node, obtaining a snapshot is relatively simple: we only need to obtain the state of all variables and combinations of events on that node at a specific time to generate the snapshot. But when the system runs with multiple nodes, things become complicated: each node has its own state, and events between nodes can affect the node's state.

An intuitive way is to use a “global clock” to notify other nodes. After receiving the message, nodes return their state before that time. As introduced earlier, synchronizing between nodes using physical clocks is unreliable. As shown in Figure 3.26, node A records its current snapshot (including events $\{A1, A2, A3\}$) at 8:10 and simultaneously sends a message to node B (the send message event is A4), asking node B to record its state. Due to clock skew, node B's time is only 8:10 after receiving the message, so the receive message event B3 is also recorded in node B's state. That is, node B's snapshot consists of events $\{B1, B2, B3\}$. Note that node A's snapshot does not include event A4, which caused event B3. Therefore, this snapshot is not consistent,

because event B3 appears out of nowhere. According to the properties that snapshots need to satisfy mentioned earlier, if event B3 appears in the snapshot, all events that happened before it should be in the snapshot, but event A4 is not in node A's snapshot, violating the snapshot property. The cause of this problem is using physical time as the basis for recording snapshots.

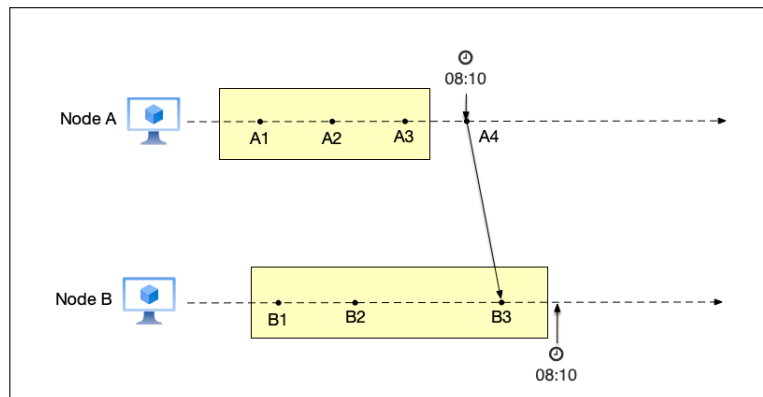


Figure 3.26: The problem of recording global state using physical time

Lamport and Chandy proposed an algorithm for recording global state in distributed systems in their paper [16] (the algorithm is named the Chandy-Lamport algorithm¹⁷ after the two authors).

In the paper, to help readers better understand the algorithm's idea, Lamport used the example of photographing a flock of birds. When a photographer wants to take a picture of a flock of birds in the sky, because there are so many birds and it is impossible to keep them still during the shot, the photographer's approach is to take multiple photos and finally stitch these photos into an overall scene. Here, birds can be analogized to nodes in a distributed system.

But the difference between a distributed system and a flock of birds is that nodes in a distributed system send messages to each other to communicate, and these messages may change the node's state. Therefore, in addition to recording the node's own state, the algorithm also needs to record the messages received by that node, so that a local snapshot of a node can be obtained. The local snapshots of all nodes in the system together form the global snapshot of the system. In addition, in order not to interfere with the message sending of nodes in the distributed system, the algorithm also defines a Marker message. This message does not change the internal state of nodes and is used to notify nodes of the start and end of recording local snapshots.

¹⁷https://en.wikipedia.org/wiki/Chandy-Lamport_algorithm

Before formally introducing the algorithm steps, we need to solve a tricky problem first: how to record those messages that are currently in flight?

Imagine we are watching a video recording of a soccer match. If at a certain second we press the pause button (snapshot), the positions of all players on the field are frozen (node state). But on the field, there may still be a soccer ball flying in the air (message in transit). It has just been kicked by player A but has not yet been received by player B.

If we only record the positions of the players and do not record this flying soccer ball, then when we resume the match (recover from the snapshot), this ball vanishes into thin air—player A thinks “I already kicked it out,” but player B has not received it, and the system state has lost data. This is obviously wrong.

Therefore, a perfect global snapshot must not only record the positions of all players (node states), but also “capture” all soccer balls flying in the air (channel messages) and put them into the snapshot.

The Chandy-Lamport algorithm cleverly uses Marker messages to solve this problem. The logic is as follows:

- Red line division (consistent cut): imagine the Marker as an invisible red dividing line.
- Before the red line: messages sent before the Marker belong to the “past” and should be included in the snapshot.
- After the red line: messages sent after the Marker belong to the “future” and should not appear in this snapshot.

When node A records its own state and sends out a Marker to node B, node A has already entered the “post-snapshot” world. However, node B has not yet received this Marker; it is still living in the “pre-snapshot” world.

During this time difference (after A sends the Marker, before B receives the Marker), if any message flies from A to B:

- For A, this message is sent after it recorded its state (belongs to the future);
- But for B, is it received before it records its state? No, the algorithm requires B to receive the Marker before it can “freeze.”

Let us look at it from the perspective of the receiver. Suppose node B has already recorded its own state (because it may have first received a Marker from C, triggering the snapshot), but it has not yet received the Marker from A. At this point, both A and B are in the process of recording the snapshot. If A sent a message $m1$ to B before sending the Marker, this message $m1$ obviously belongs to the “past.”

- If $m1$ arrives before B records its state, B will update its own state ($x=x+m1$), which is

fine.

- Key point: if m1 flies slowly and arrives after B has already recorded its local state (taken the photo), but at this time B has not yet received A's Marker. This means m1 belongs to "before the red line" (A sent it before sending the Marker), but it missed B's "photo moment."

In order not to lose this "late" m1, the algorithm requires: after node B records its own state, it must turn on a video recorder to specifically record all messages coming from A's channel until it receives A's Marker. These messages are the "patch data" that "clearly belong to the past but missed the photo due to network delay."

Understanding that recording channel messages is to capture those late, in-transit messages belonging to the past state, let us look at the specific algorithm flow. The algorithm has the following assumptions:

- Messages between nodes will not be lost and will eventually arrive at the receiver;
- Message channels between nodes are FIFO queues, i.e., messages will not be out of order;
- Message channels on nodes are divided into two types: outbound channels and inbound channels;
- Nodes will not crash.

With the above assumptions, let us look at the algorithm implementation.

1. The node responsible for initializing the snapshot has the following tasks:
 - Record the current state of this node;
 - Broadcast marker messages to other nodes through the outbound channels on this node;
 - Start recording messages received on inbound channels;
2. If a node that has started recording a snapshot receives a message that is not a marker message, it needs to record these messages until it receives a marker message;
3. After a node (including the node responsible for initializing the snapshot) receives a marker message, depending on whether it has received a marker message before, there are different treatments:
 - If it has not received a marker message before:
 - The node starts recording the state on this node;
 - Mark the channel that received the marker message as empty;

- Broadcast marker messages to other nodes through the outbound channels on this node.
 - If it has received a marker message before, stop recording events from the channel that received the marker message.
4. When all nodes' inbound channels have received marker messages, it means all nodes have recorded their snapshots. The local snapshots on all nodes will form a global snapshot.

Let us use an example to illustrate the operation flow of the Chandy-Lamport algorithm:

1. Initial state As shown in Figure 3.27, the system has two nodes A and B, with states $\{x = 1\}$ and $\{y = 2\}$ respectively;

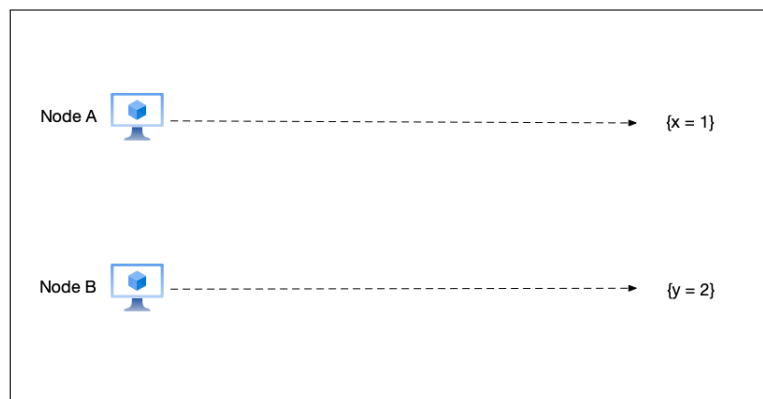


Figure 3.27: Chandy-Lamport algorithm example 1: initial state

2. Start recording snapshot As shown in Figure 3.28, node A records the current snapshot of this node as $\{x = 1\}$, then sends a marker message to node B, notifying it to start recording a snapshot. Before node B receives the marker message, node B also sends a data modification message to node A.
3. Record received non-marker events As shown in Figure 3.29, node A receives the data modification message from node B. Since this message is not a marker message, node A needs to record this message. After node B receives the marker message from node A, it needs to record the current snapshot as $\{y = 2\}$.
4. Complete global snapshot recording As shown in Figure 3.30, after recording the snapshot, node B sends a marker message to node A. After node A receives the marker message

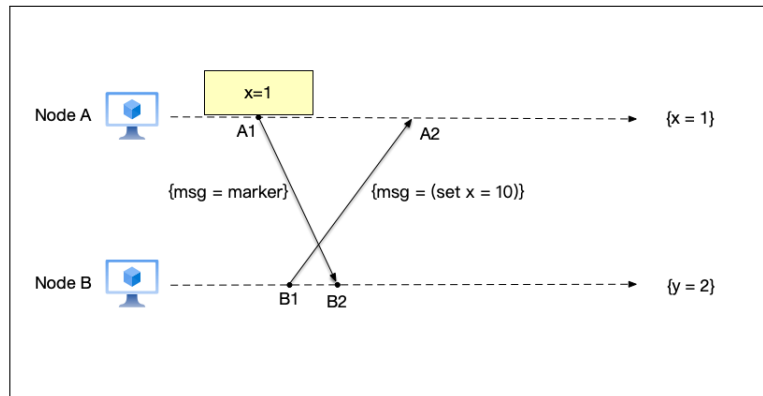


Figure 3.28: Chandy-Lamport algorithm example 2: start recording snapshot

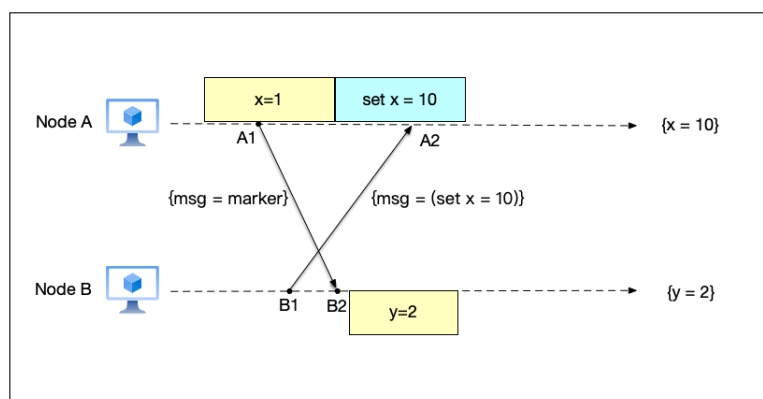


Figure 3.29: Chandy-Lamport algorithm example 3: node A records other received events

from node B, it knows that node B has completed its local snapshot. Since there are only two nodes in this system, a global snapshot is generated at this point. The local snapshots are: snapshot $\{x = 1\}$ and event $\text{set } x = 10$ on node A, and snapshot $\{y = 2\}$ on node B. Together, these local snapshots form the global snapshot $\{x = 10, y = 2\}$.

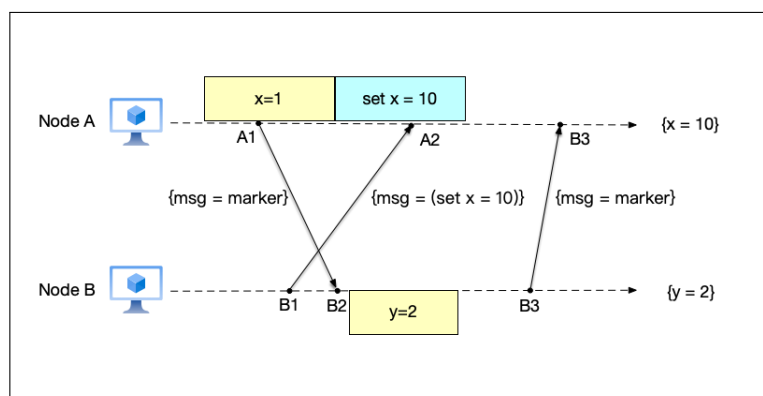


Figure 3.30: Chandy-Lamport algorithm example 4: complete global snapshot recording

3.8 Chapter Summary

In this chapter, we started from the most fundamental yet most confusing concept in distributed systems—“time”—and explored how to determine the order of events in a system without a global clock. We first recognized that order determines state. If a distributed system is viewed as a state machine, then only by ensuring that each replica executes the same events in the same order can it ultimately reach a consistent state. However, attempting to find this order through physical clocks is unreliable. Whether it is the wall clock or the monotonic clock, limited by the physical characteristics of quartz oscillation, the network asymmetry of the NTP protocol, and the accumulated error of hierarchical synchronization, physical time always has skew and drift during cross-node communication and cannot serve as an absolute measure for determining the order of events. To break free from the constraints of physical time, we introduced Lamport’s Happened-before relation ($\rightarrow$), which is the cornerstone for understanding distributed ordering. It divides the relationships between events into “before” relationships with causality and “concurrent” relationships that cannot be compared. Based on this, we distinguished the two important mathematical concepts of total order and partial order: single-node systems usually satisfy total order, while distributed systems are essentially partial order systems after concurrency is introduced. In the exploration of logical clocks, we introduced two implementations:

- Lamport clocks: using a simple counter to satisfy the basic clock condition ($a \rightarrow b \Rightarrow C(a) < C(b)$). It imposes a total order on all events but loses some causal information and cannot infer event relationships in reverse.
- Vector clocks: by maintaining causal history, it achieves the strong consistency condition ($a \rightarrow b \iff V(a) < V(b)$). It can accurately identify concurrent events, but at the cost of storage and network overhead that grows linearly with the number of nodes.

Finally, as an important application of logical clocks, we learned the Chandy-Lamport global snapshot algorithm. This algorithm cleverly uses Marker messages as “dividing lines” between nodes, capturing a globally consistent state (global state) that satisfies causal consistency without stopping the system’s operation (non-blocking). This tells us that a “snapshot” in a distributed system is not the same moment in physical time, but a logically consistent cut.

Through the study of this chapter, we have established a core concept: in distributed systems, causality is more important than physical time.

By understanding “time and order”, we unlock the door to distributed systems. In the following chapters, we will see how these theories are transformed into engineering practice:

from data replication between replicas, to resolving concurrent conflicts, to implementing consensus algorithms (such as Paxos and Raft), all of which rely on the foundational theories about “order” established in this chapter.

Bibliography

- [1] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung: “[The Google File System](#),” at 19th ACM Symposium on Operating Systems Principles (SOSP), October 2003. [doi:10.1145/945445.945450](#)
- [2] Martin Kleppmann: “Designing Data-Intensive Applications,” 1st edition. O’Reilly Media, 2017.
- [3] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair: “Distributed Systems: Concepts and Design,” 5th edition. Pearson, 2011.
- [4] W. Richard Stevens: “TCP/IP Illustrated, Volume 1: The Protocols,” Addison-Wesley, 1994.
- [5] E.A. Akkoyunlu, K. Ekanadham, and R.V. Huber: “[Some Constraints and Tradeoffs in the Design of Network Communications](#),” at Proceedings of the Fifth ACM Symposium on Operating Systems Principles (SOSP ’75), Austin, Texas, USA, 1975.
- [6] Jim Gray: “[Notes on Data Base Operating Systems](#),” 1978.
- [7] Leslie Lamport, Robert Shostak, and Marshall Pease: “[The Byzantine Generals Problem](#),” ACM Transactions on Programming Languages and Systems (TOPLAS), volume 4, number 3, pages 382–401, July 1982. [doi:10.1145/357172.357176](#)

- [8] Cynthia Dwork, Nancy A. Lynch, and Larry Stockmeyer: “[Consensus in the Presence of Partial Synchrony](#),” *Journal of the ACM*, volume 35, number 2, pages 288–323, April 1988. doi:10.1145/42282.42283
- [9] Peter Bailis, Kyle Kingsbury: “[The Network Is Reliable](#),” *ACM Queue*, volume 12, number 7, pages 20–32, July 2014. doi:10.1145/2639988.2655736
- [10] Satoshi Nakamoto: “[Bitcoin: A Peer-to-Peer Electronic Cash System](#),” 2008.
- [11] Steven Allen: “[Planes Will Crash! Things That Leap Seconds Didn’t, and Did, Cause](#),” 2013.
- [12] Paul R. Johnson and Robert H. Thomas: “[RFC 677: Maintenance of Duplicate Databases](#),” Network Working Group, January 1975.
- [13] Leslie Lamport: “[Time, Clocks, and the Ordering of Events in a Distributed System](#),” *Communications of the ACM*, volume 21, number 7, pages 558–565, July 1978. doi:10.1145/359545.359563
- [14] M. Raynal and M. Singhal: “[Logical Time: Capturing Causality in Distributed Systems](#),” *IEEE Computer*, volume 29, number 2, pages 49–56, February 1996.
- [15] Friedemann Mattern: “[Virtual Time and Global States of Distributed Systems](#),” in *Proceedings of the Workshop on Parallel and Distributed Algorithms*, pages 215–226, North-Holland, 1989.
- [16] K. Mani Chandy and Leslie Lamport: “[Distributed Snapshots: Determining Global States of Distributed Systems](#),” *ACM Transactions on Computer Systems (TOCS)*, volume 3, number 1, pages 63–75, February 1985.
- [17] Yoshinori Matsunobu: “[Semi-Synchronous Replication at Facebook](#),” yoshinorimat-sunobu.blogspot.com, 2014.
- [18] Jay Kreps: “[I Heart Logs](#),” O’Reilly Media, 2014.
- [19] ACM Queue: “[A Conversation with Bruce Lindsay](#),” *ACM Queue*, 2004.
- [20] Leslie Lamport: “[How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs](#),” *IEEE Transactions on Computers*, volume 28, number 9, pages 690–691, September 1979.

-
- [21] Maurice P. Herlihy and Jeannette M. Wing: “[Linearizability: A Correctness Condition for Concurrent Objects](#),” ACM Transactions on Programming Languages and Systems (TOPLAS), volume 12, number 3, pages 463–492, July 1990.
 - [22] Leslie Lamport: “[On Interprocess Communication](#),” Distributed Computing, volume 1, numbers 2 and 3, 1986.
 - [23] Douglas B. Terry, Alan J. Demers, Karin Petersen, Mike Spreitzer, Marvin Theimer, and Brent Welch: “[Session Guarantees for Weakly Consistent Replicated Data](#),” at 3rd International Conference on Parallel and Distributed Information Systems (PDIS), September 1994.
 - [24] Werner Vogels: “[Eventually Consistent](#),” ACM Queue, 2007.
 - [25] Werner Vogels: “[Eventually Consistent — Revisited](#),” allthingsdistributed.com, 2008.
 - [26] Bowen Alpern and Fred B. Schneider: “[Defining Liveness](#),” Information Processing Letters, volume 21, number 4, pages 181–185, October 1985.
 - [27] Doug Terry: “[Replicated Data Consistency Explained Through Baseball](#),” ACM Queue, volume 11, number 5, pages 28–37, May 2013.
 - [28] David K. Gifford: “[Weighted Voting for Replicated Data](#),” at 7th ACM Symposium on Operating Systems Principles (SOSP), 1979.
 - [29] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al.: “[Dynamo: Amazon’s Highly Available Key-value Store](#),” at 21st ACM Symposium on Operating Systems Principles (SOSP), October 2007.
 - [30] Michael Whittaker, Aleksey Charapko, Joseph M. Hellerstein, Heidi Howard, Ion Stoica: “[Read-Write Quorum Systems Made Practical](#),” 2021.
 - [31] Michael J. Fischer, Nancy Lynch, and Michael S. Paterson: “[Impossibility of Distributed Consensus with One Faulty Process](#),” Journal of the ACM, volume 32, number 2, pages 374–382, April 1985.
 - [32] Paper Trail: “[A Brief Tour of FLP Impossibility](#),” the-paper-trail.org, 2008.
 - [33] Leslie Lamport: “[The Part-Time Parliament](#),” ACM Transactions on Computer Systems (TOCS), volume 16, number 2, pages 133–169, May 1998.

- [34] Leslie Lamport: [“Paxos Made Simple,”](#) ACM SIGACT News, volume 32, number 4, pages 18–25, December 2001.
- [35] Mike Burrows: [“The Chubby Lock Service for Loosely-Coupled Distributed Systems,”](#) at 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2006.
- [36] Tushar Chandra, Robert Griesemer, and Joshua Redstone: [“Paxos Made Live — An Engineering Perspective,”](#) at ACM Symposium on Principles of Distributed Computing (PODC), 2007.
- [37] Zhiying Liang, Vahab Jabrayilov, and Aleksey Charapko: [“MultiPaxos Made Complete,”](#) 2024.
- [38] Diego Ongaro and John K. Ousterhout: [“In Search of an Understandable Consensus Algorithm,”](#) at USENIX Annual Technical Conference, 2014.
- [39] Diego Ongaro: [“Consensus: Bridging Theory and Practice,”](#) Stanford University PhD Thesis, 2014.
- [40] David Karger, Eric Lehman, Tom Leighton, et al.: [“Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web,”](#) at 29th Annual ACM Symposium on Theory of Computing (STOC), 1997.
- [41] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, Hari Balakrishnan: [“Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications,”](#) at ACM SIGCOMM, August 2001.
- [42] Werner Vogels: [“A Decade of Dynamo,”](#) allthingsdistributed.com, 2017.
- [43] Ralph C. Merkle: [“A Digital Signature Based on a Conventional Encryption Function,”](#) in Advances in Cryptology — CRYPTO ’87, LNCS 293, pages 369–378, 1987.
- [44] Theo Haerder, Andreas Reuter: [“Principles of Transaction-Oriented Database Recovery,”](#) ACM Transactions on Database Systems (TODS), volume 9, number 4, pages 540–565, December 1983.
- [45] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil: [“A Critique of ANSI SQL Isolation Levels,”](#) Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data, pages 1–10, May 1995.

-
- [46] H.T. Kung, J.T. Robinson: “[On Optimistic Methods for Concurrency Control](#),” ACM Transactions on Database Systems (TODS), volume 6, number 2, pages 213–226, June 1981.
 - [47] David Patrick Reed: “[Naming and Synchronization in a Decentralized Computer System](#),” MIT PhD Thesis, 1978.
 - [48] Philip A. Bernstein and Nathan Goodman: “[Concurrency Control in Distributed Database Systems](#),” ACM Computing Surveys, volume 13, number 2, pages 185–221, June 1981.
 - [49] Yingjun Wu, Joy Arulraj, et al.: “[An Empirical Evaluation of In-Memory Multi-Version Concurrency Control](#),” VLDB, 2017.
 - [50] Dan R. K. Ports and Kevin Grittnr: “[Serializable Snapshot Isolation in PostgreSQL](#),” Proceedings of the VLDB Endowment, volume 5, number 12, pages 1850–1861, August 2012.
 - [51] James C. Corbett, Jeffrey Dean, Michael Epstein, et al.: “[Spanner: Google’s Globally-Distributed Database](#),” at 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2012.
 - [52] Keith Marzullo, Susan Owicki: “[Maintaining the Time in a Distributed System](#),” Stanford University, CSL Technical Report 83–247, 1983.
 - [53] Eric A. Brewer: “[Towards Robust Distributed Systems](#),” at ACM PODC Keynote, 2000.
 - [54] Eric A. Brewer: “[CAP Twelve Years Later: How the Rules Have Changed](#),” infoq.com, 2012.
 - [55] Seth Gilbert and Nancy Lynch: “[Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services](#),” ACM SIGACT News, volume 33, number 2, pages 51–59, June 2002.
 - [56] Daniel J. Abadi: “[Problems with CAP, and Yahoo’s Little Known NoSQL System](#),” dbmsmusings.blogspot.com, 2010.
 - [57] Daniel J. Abadi: “[Consistency Tradeoffs in Modern Distributed Database System Design](#),” IEEE Computer, volume 45, number 2, pages 44–50, 2012.
 - [58] Martin Kleppmann: “[A Critique of the CAP Theorem](#),” 2015.

- [59] Dan Pritchett: "[BASE: An Acid Alternative](#)," ACM Queue, 2008.
- [60] Pat Helland: "[Life Beyond Distributed Transactions: An Apostate's Opinion](#)," ACM Queue, 2007.
- [61] Hector Garcia-Molina, Kenneth Salem: "[SAGAS](#)," at ACM SIGMOD, 1987.